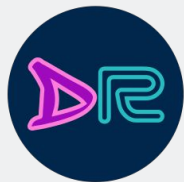


ROB 430/599: Deep Learning for Robot Perception and Manipulation (DeepRob)

Lecture 20: 3D Vision; Geometric Foundation Models

03/30/2026



<https://deeprob.org/w26/>

Tomorrow: Project Lightning Talk

Tuesday March 31, 2026

Starts 3:30pm EST

- We will go by group number order (1 - 20)
 - 5 min presentation + 1 min Q&A (hard cutoff)
 - All group members must speak
- Special request regarding order?
 - Please post a private Piazza post by end of today
- Upload presentation files to [Final Project LightningTalk folder](#) on Google Drive by March 31 3:30pm EST
 - Filename: "lightning_talk_group[id]"
- 5% grade

Group 1	Changhe Chen Chi Zhang Peng-Chen Chen Yifan Chou	changhe zhc pochenm yfchou	diffusion policy, tabletop manipulation
Group 2	Anna Novak Anuha Tao Mark Gibney Brendan Ohr	annanova anuha msgibney bdanohr	
Group 3	Rui Zhang Zijie Chen Jinhao Yu Xinyuan Wang	zrui chenzj rickyu wxinyuan	reactive diffusion policy, tactile, async inference
Group 4	Karthik Vairavan Avantika Rattan Saumit Vedula	arattan arattan	
Group 5	Akash Sarada Ashrith Edukulla Elias Fandi Lara Lim	asarada ashed efandi laralim	
Group 6	Sebastian Helges Ryan Tsai Sam Vance Zach Charlick	sebhelge ryantsai svance zsc	
Group 7	Cody Sheltraw Tate Collier Tisha Jain Halina Costello	cshel tatecoll tishaj halinac	
Group 8	Yijie Liao Weiye Wang Miaomiao Dai Pailin Meng	lyjie wweiyi dmmeju pallen	
Group 9	Shicheng Peng Zehan Qin Felicia Sang	shchpeng zehanqin fsang	
Group 10	yi wang jiawei liu Henry Vergowen Noah Dujovny	yiiwang jiawei liu henryv dujovny	Idea 1. 360° video of an object -> Grounding DINO + SAM to segment -> frame extraction (where object stays in one pose) -> COLMAP to generate 3D pose info -> instant-rgp (NeRF) -> NeRF to mesh. Personally, I'm also interested in extracting anime-style characters from 3D games as a fun application. Idea 2. Control related project.
Group 11	Yuan Ren Lucas Swierad Ben Taylor	yuanren lswierad bmtay	
Group 12	Taimafei Lazowski Min Kyu Kim	tiazousk minqkim	marine mapping
Group 13	Rishad Hasan Tetsu Kurumisawa Athanva Kashyap	rishadh tetsuk katharva	VLA's, neurosymbolic models, relational graphs
Group 14	Yuri Carnino Nithin Reddy Robert Roe	ycarnino nkreddy rwroe	Mobile manipulation, VLA's
Group 15	Ashley Ruder Will Mahoney	ander wmahoney	
Group 16	Guangxuan Xu	eggsyxu	End-to-End Autonomous Drone Target Tracking using Vision
Group 17	Syeda Reza Darren Cleeman ethan LIN	syerez dcleeman YSLIN	Depth Anything V2 for Robot Navigation
Group 18	Ken Erika Chen Elaina Mann	tzuchieh ERIKACHN ELAINAMN	NeRF SLAM
Group 19	Ruey Day	rueyday	3D Line segmentation
Group 20	Junwoon Lee		Neural Representation

Recap: ViT and CNN

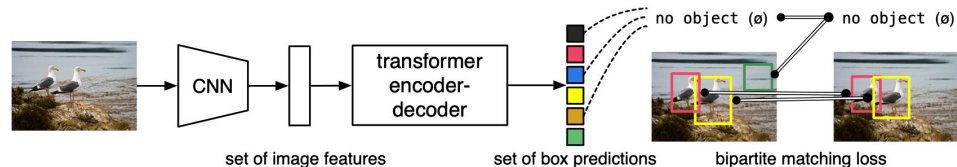
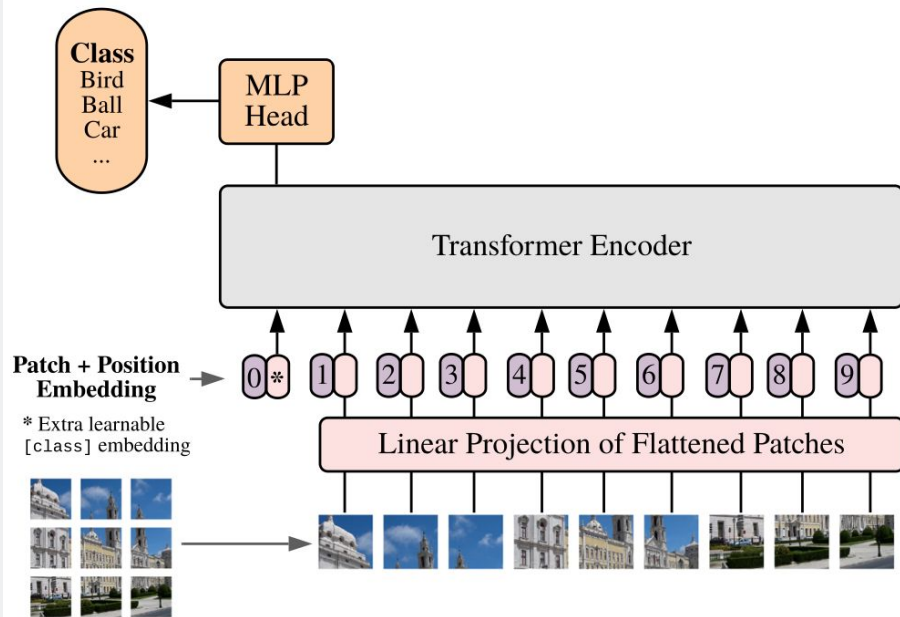


Fig. 1: DETR directly predicts (in parallel) the final set of detections by combining a common CNN with a transformer architecture. During training, bipartite matching uniquely assigns predictions with ground truth boxes. Prediction with no match should yield a “no object” ($\emptyset$) class prediction.

Recap: Vision Mamba

<https://arxiv.org/pdf/2401.09417>

2D vision task

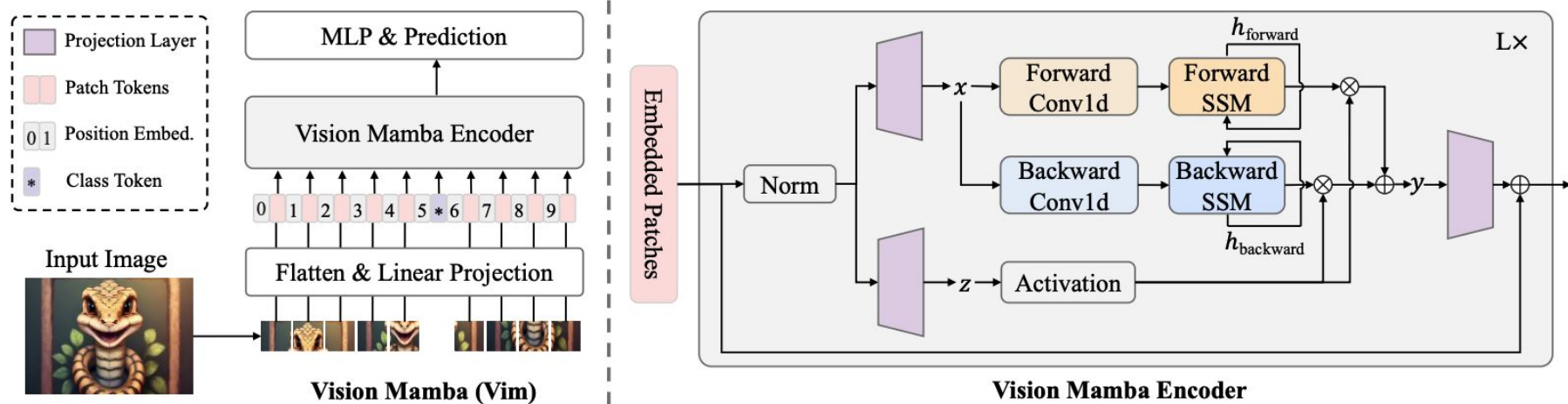
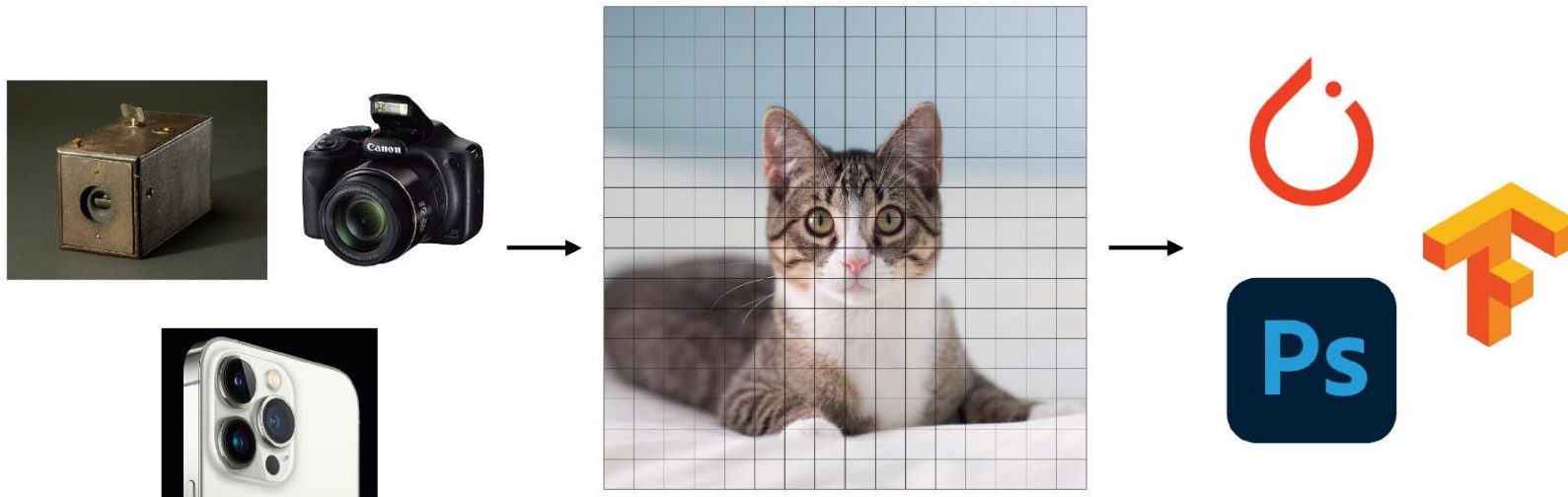


Figure 2: The overview of the proposed Vim model. We first split the input image into patches, and then project them into patch tokens. Last, we send the sequence of tokens to the proposed Vim encoder. To perform ImageNet classification, we concatenate an extra learnable classification token to the patch token sequence. Different from Mamba for text sequence modeling, Vim encoder processes the token sequence with both forward and backward directions.

Today

- Representations and Learning for 3D Data
- A Primer on 3D Reconstruction
- Summary and Takeaways

2D Representation



Unified representation across
applications, tools, and sensors

Meanwhile, in 3D ...

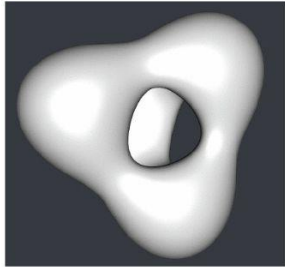
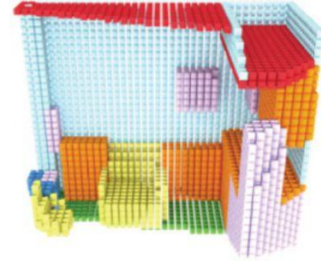
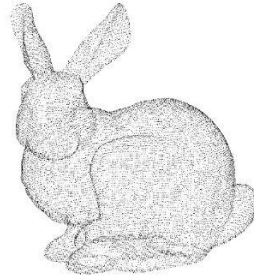
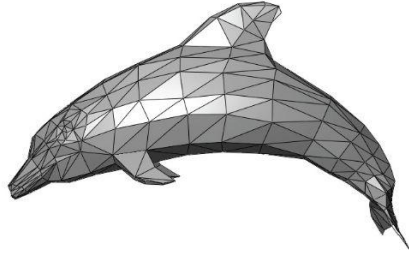
Ahaslide: what is your favorite way to represent **3D data**? (Example: point cloud, ...)



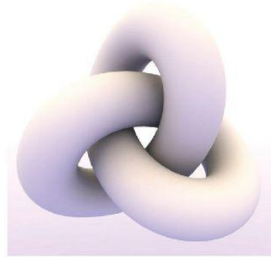
<https://ahaslides.com/VD9KY>

Today: 3D representations

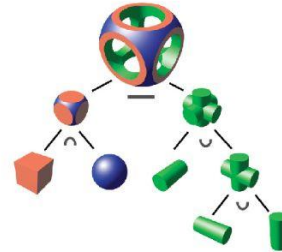
And how they are useful for robot (learning)



$$\{\mathbf{p} \mid f(\mathbf{p}) = 0\}$$



$$f(\mathbf{u}) = \mathbf{p} \in \mathbb{R}^3$$



and many more ...

Depth Map



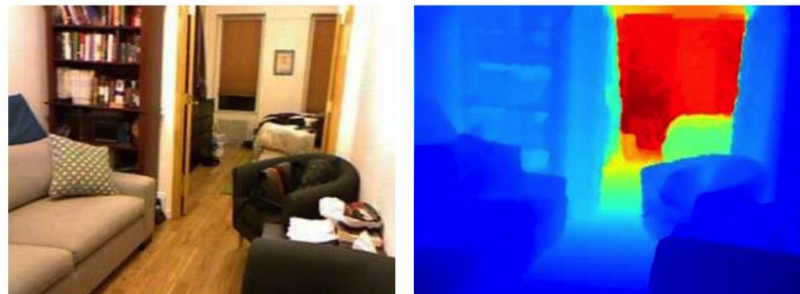
Common modality across sensors

Depth Map

For each pixel, **depth map** gives distance from the camera to the object in the world at that pixel

RGB image + Depth image
= RGB-D Image (2.5D)

This type of data can be recorded directly for some types of 3D sensors (e.g. Microsoft Kinect)



RGB Image: $3 \times H \times W$ Depth Map: $H \times W$

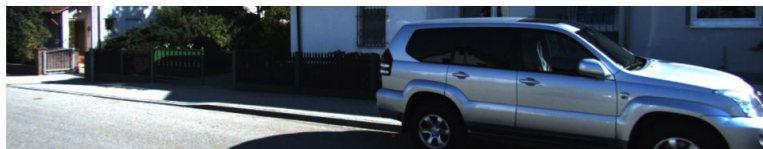
https://openaccess.thecvf.com/content_iccv_2015/papers/Eigen_Predicting_Depth_Surface_ICCV_2015_paper.pdf

Predicting Depth Maps

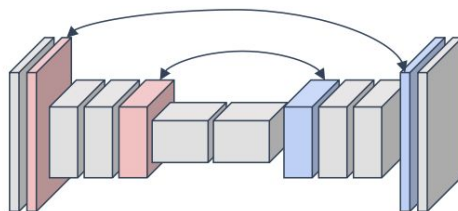
(KITTI dataset)

https://papers.nips.cc/paper_files/paper/2014/hash/91c56ce4a249fae5419b90cba831e303-Abstract.html

<https://www.cvlibs.net/datasets/kitti/>



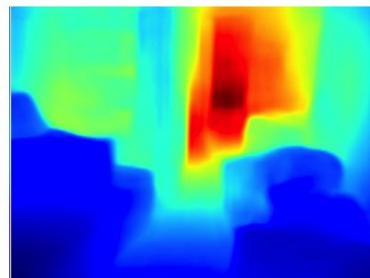
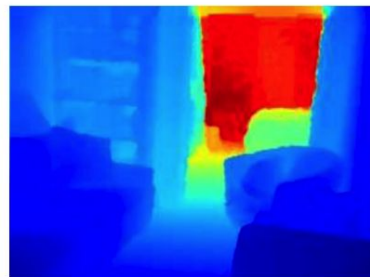
RGB Input Image:
 $3 \times H \times W$



**Fully Convolutional
network**

Predicted Depth Image:

$1 \times H \times W$



Predicted Depth Image:
 $1 \times H \times W$

**Per-Pixel Loss
(L2 Distance)**

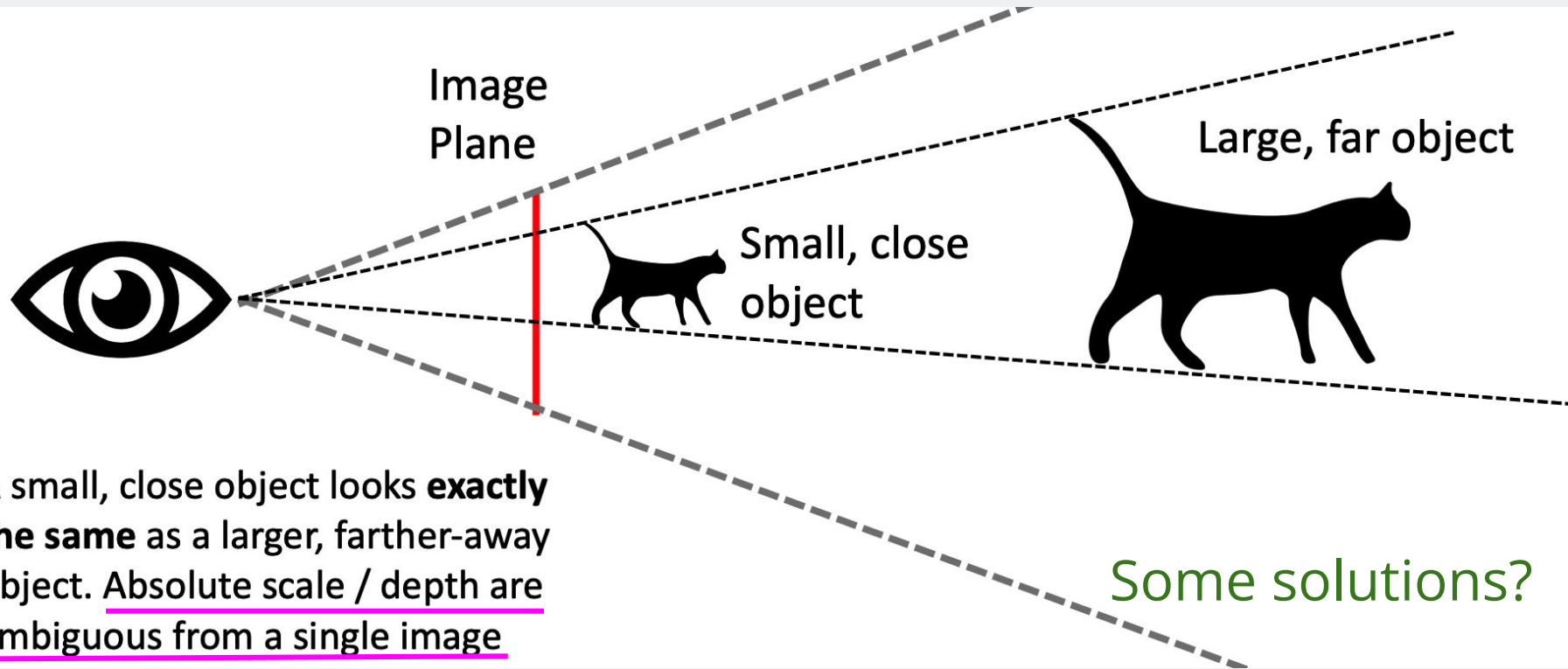
Ahaslide

What are potential challenges of learning to predict depth from an RGB image?



<https://ahaslides.com/VD9KY>

Problem: Scale / Depth Ambiguity



Predicting Depth Maps

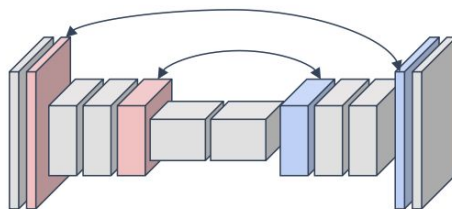
$$d_i = \log y_i - \log y_i^*.$$

Scale invariant loss

$$\begin{aligned} D(y, y^*) &= \frac{1}{2n^2} \sum_{i,j} ((\log y_i - \log y_j) - (\log y_i^* - \log y_j^*))^2 \\ &= \frac{1}{n} \sum_i d_i^2 - \frac{1}{n^2} \sum_{i,j} d_i d_j = \frac{1}{n} \sum_i d_i^2 - \frac{1}{n^2} \left(\sum_i d_i \right)^2 \end{aligned}$$

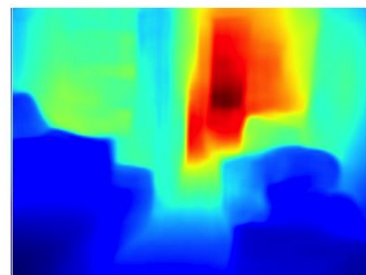
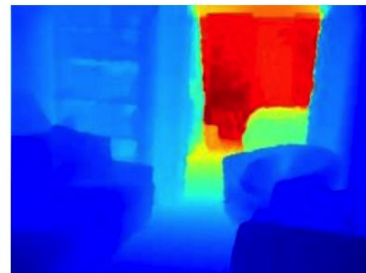


RGB Input Image:
3 x H x W



**Fully Convolutional
network**

Predicted Depth Image:
1 x H x W



**Per-Pixel Loss
(Scale invariant)**

3D Shape Representations: Surface Normals

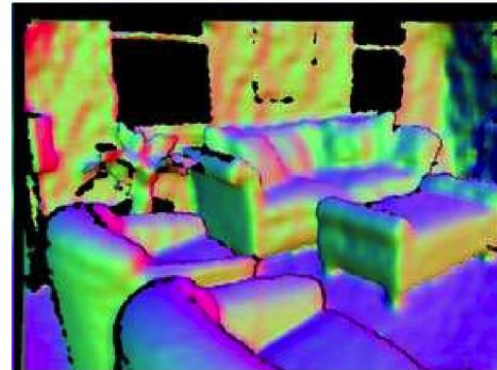
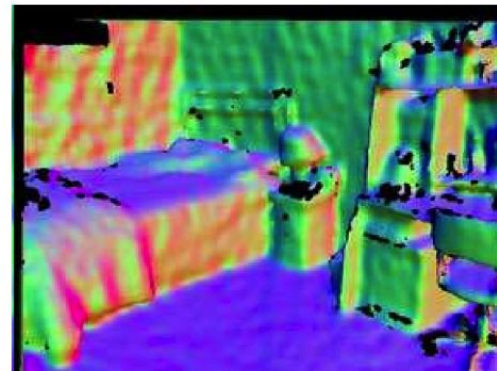
For each pixel, **surface normals** give a vector giving the normal vector to the object in the world for that pixel

Q: How to obtain surface normals?

<https://arxiv.org/pdf/1411.4734>



RGB Image: $3 \times H \times W$

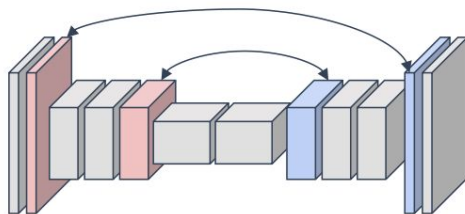


Normals: $3 \times H \times W$

Predicting Surface Normals



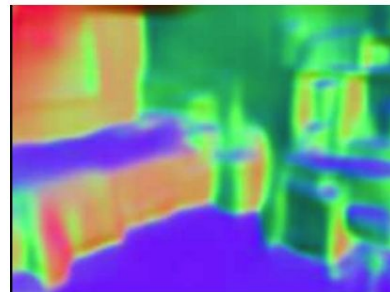
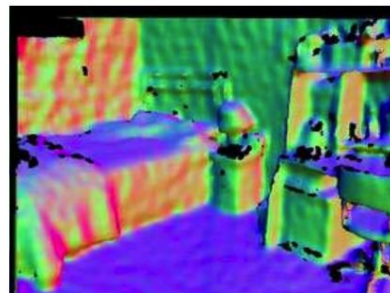
RGB Input Image:
 $3 \times H \times W$



Fully Convolutional network

Ground-truth Normals:

$3 \times H \times W$



Predicted Normals:
 $3 \times H \times W$

Per-Pixel Loss:
 $(x \cdot y) / (|x| |y|)$

Recall:

$$\begin{aligned} & x \cdot y \\ &= |x| |y| \cos \theta \end{aligned}$$

Demo: Open3D



News

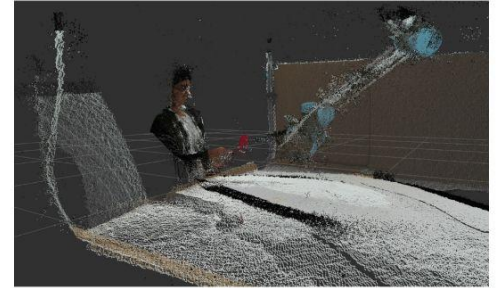
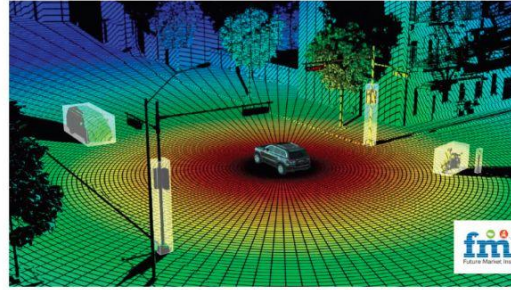
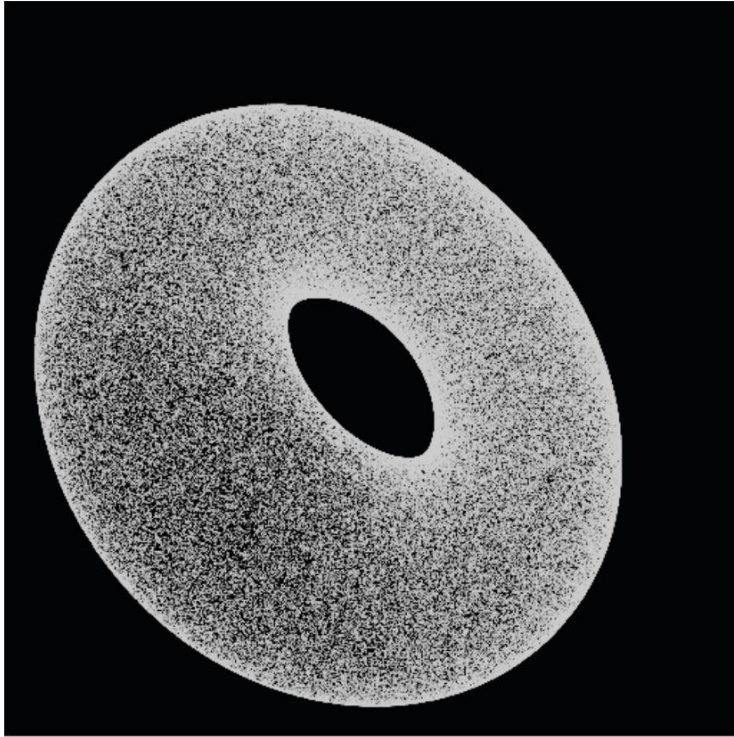
- [Open3D v0.19 is out with new features and more GPU support!](#) January 9, 2025
- [Start your new year with Open3D v0.18!](#) January 4, 2024
- [Open3D 0.17 is out!](#) March 15, 2023
- [Open3D 0.16 is out!](#) October 19, 2022
- [Open3D 0.15: more than 500+ issues addressed](#) March 1, 2022

GET IN TOUCH!



Subscribe!

3D Point Cloud

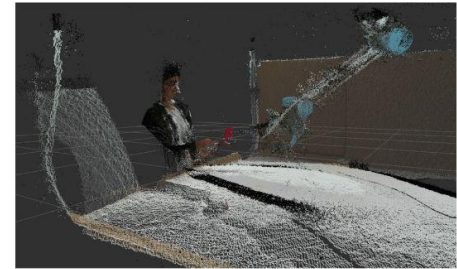
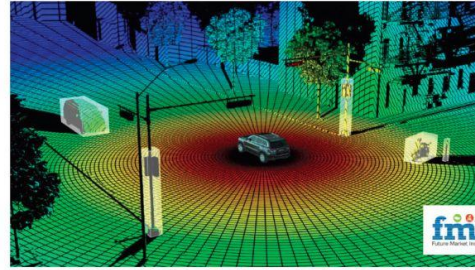


3D Point Cloud

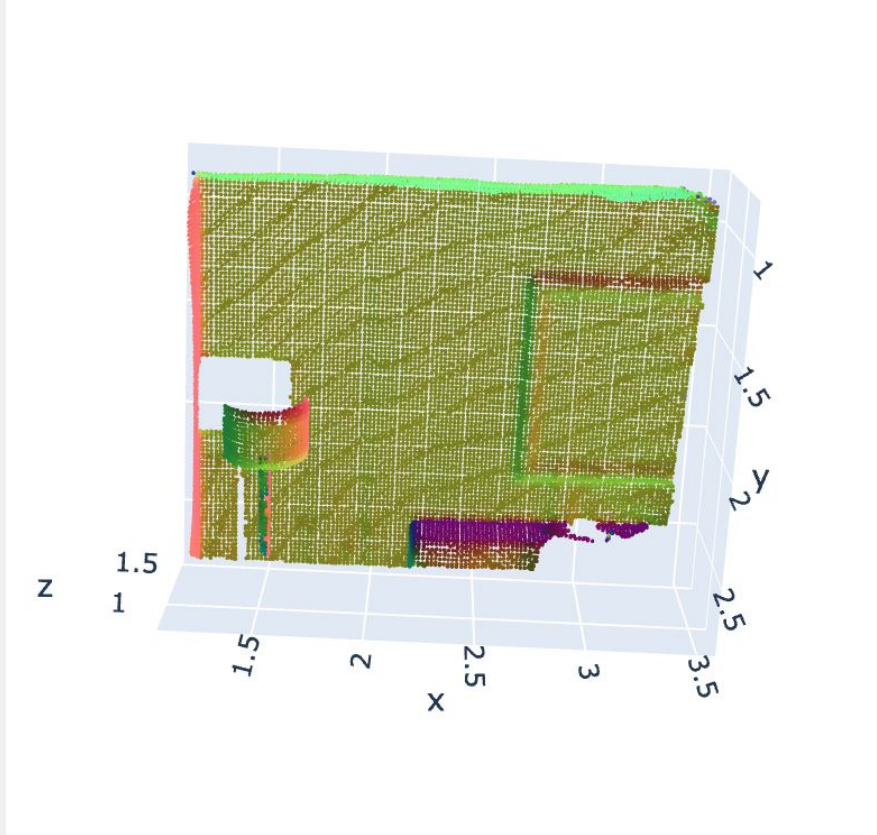
Often acquired in an image-like manner from sensors but can be useful to 'forget' this

e.g. point clouds from multiple views can make a single point cloud

can help unify approaches independent of sensing modality

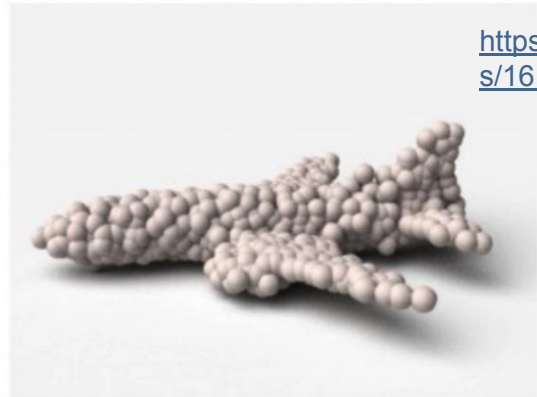
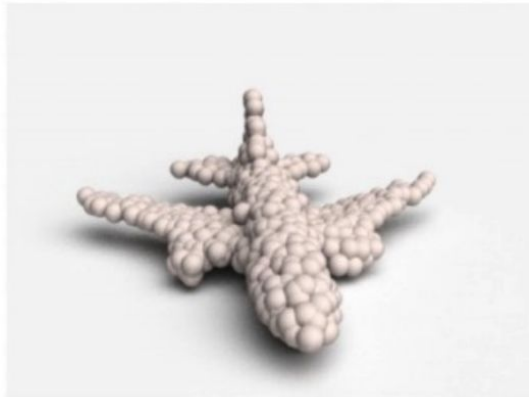


Demo: Open3D



Learning on 3D Point Cloud

- Represent shape as a set of P points in 3D space
- (+) Can represent fine structures without huge numbers of points
- () Requires new architecture, losses, etc
- (-) Doesn't explicitly represent the surface of the shape: extracting a mesh for rendering or other applications requires post-processing



<https://arxiv.org/abs/1612.00603>

PointNet

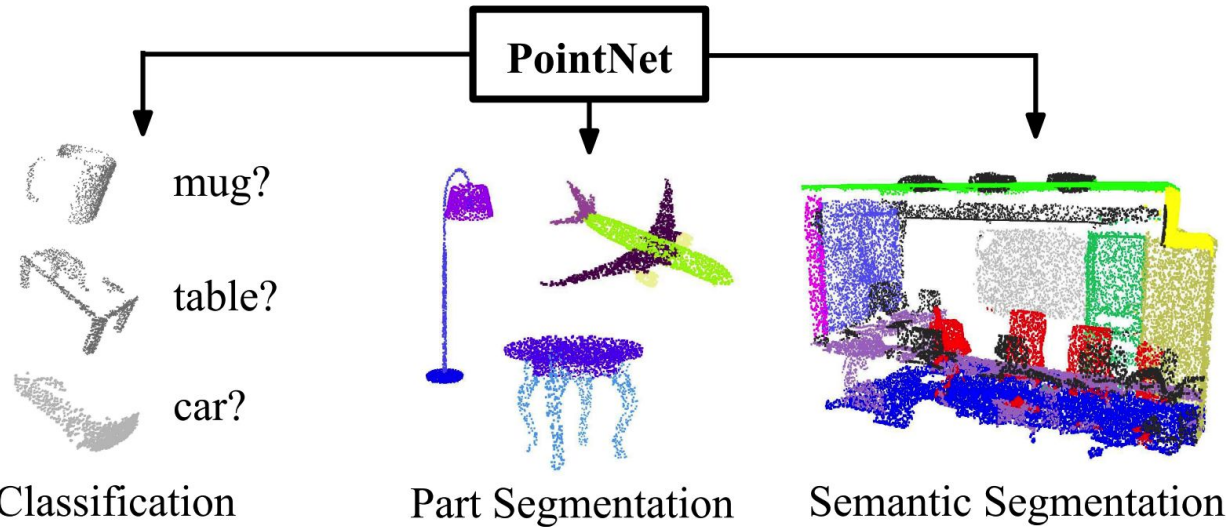
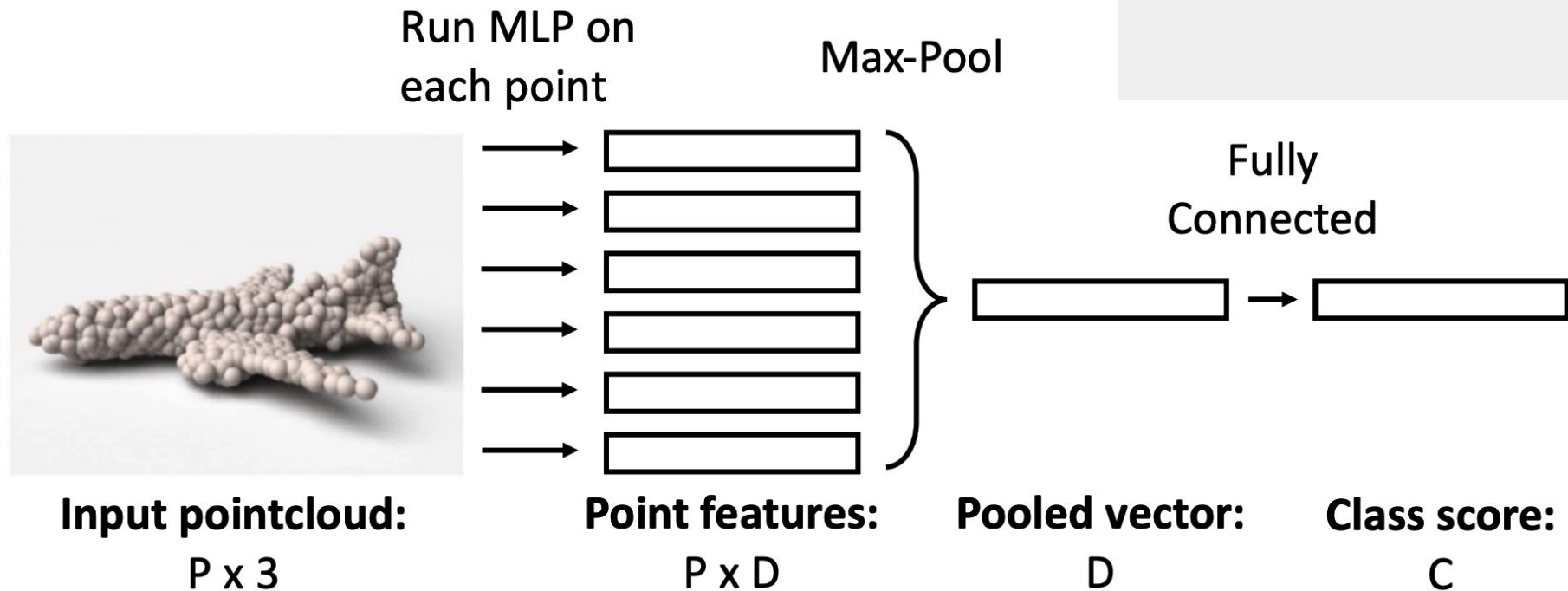


Figure 1. Applications of PointNet. We propose a novel deep net architecture that consumes raw point cloud (set of points) without voxelization or rendering. It is a unified architecture that learns both global and local point features, providing a simple, efficient and effective approach for a number of 3D recognition tasks.

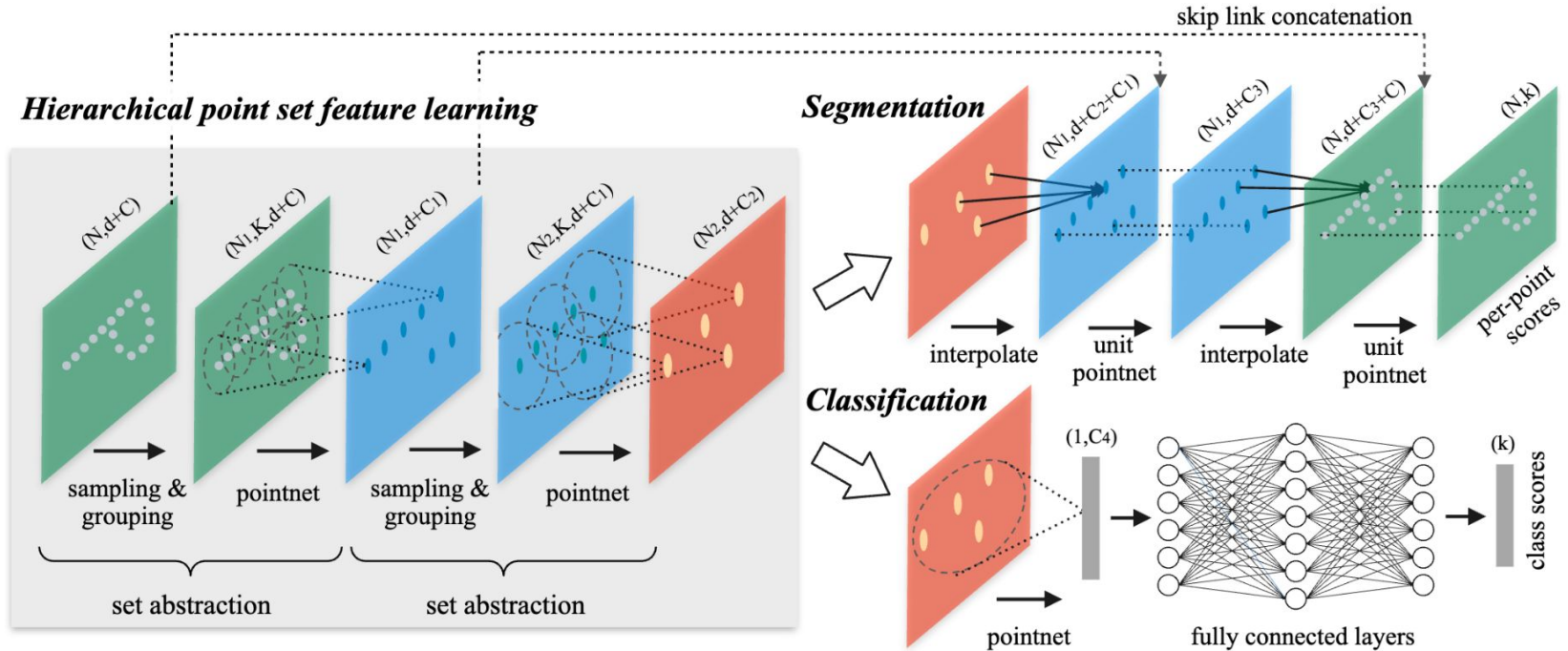
https://openaccess.thecvf.com/content_cvpr_2017/papers/Qi_PointNet_Deep_Learning_CVPR_2017_paper.pdf

PointNet

PointNet is **permutation equivariant!**

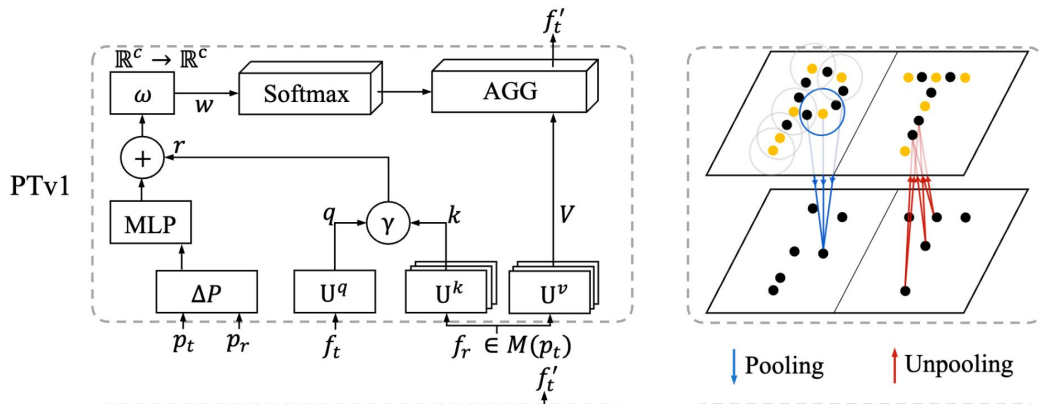


PointNet++: multiscale feature aggregation

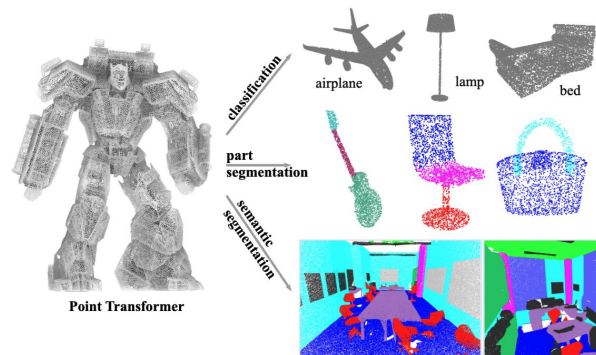


PointTransformer

<https://arxiv.org/pdf/2210.05666>

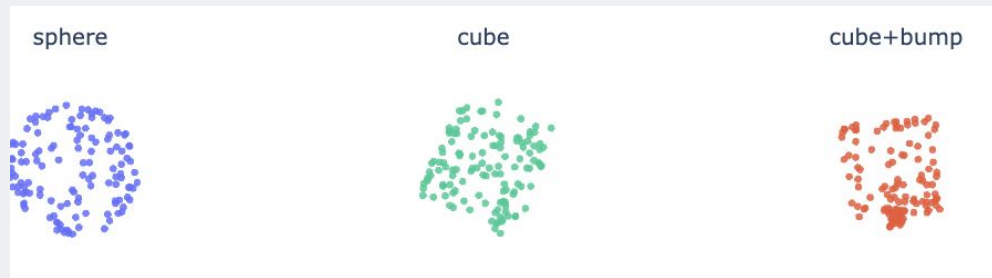


$$\begin{aligned}
 q_i &= U^q x_i, \quad k_j = U^k x_j, \quad v_j = U^v x_j, \\
 r_{ij} &= \theta(p_i - p_j), \\
 a_{ij} &= \text{Softmax}_{j \in \mathcal{N}(i)} (\gamma(q_i - k_j + r_{ij})), \\
 x'_i &= \sum_{j \in \mathcal{N}(i)} a_{ij} \odot (v_j + r_{ij}).
 \end{aligned}$$



In Class Activity: PointNet vs. PointTransformer

Which model might perform better for this point cloud classification task? Why?

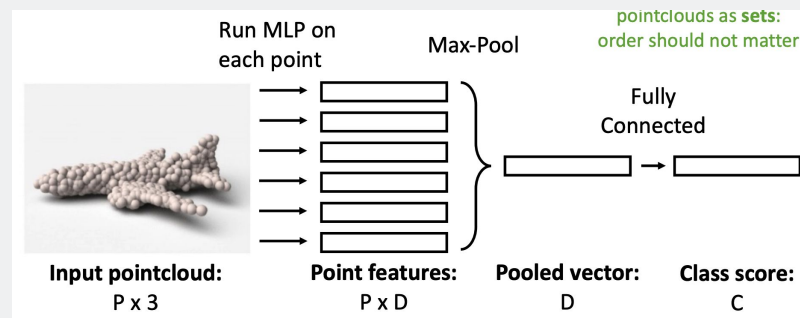


<https://ahaslides.com/VD9KY>

In Class Activity: PointNet vs. PointTransformer

```
class PointNetSmall(nn.Module):
    def __init__(self, num_classes=NUM_CLASSES):
        super().__init__()
        self.mlp = nn.Sequential(
            nn.Linear(3, 8), nn.ReLU(),
            nn.Linear(8, 16), nn.ReLU(),
            nn.Linear(16, 16), nn.ReLU(),
        )
        self.head = nn.Sequential(
            nn.Linear(16, 8), nn.ReLU(),
            nn.Dropout(0.2),
            nn.Linear(8, num_classes),
        )

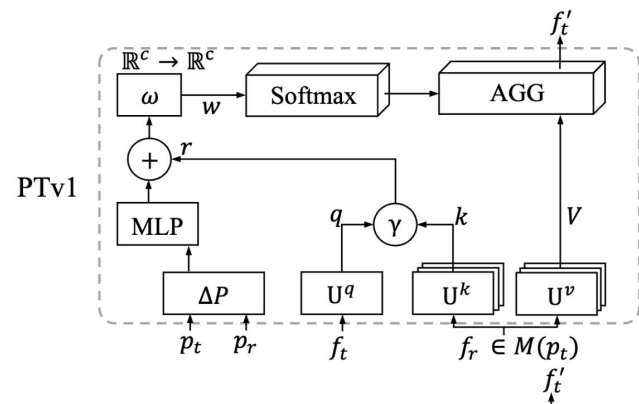
    def forward(self, pos):
        x = self.mlp(pos) # Feature encoding for each point
        global_feat = x.max(dim=1).values # Global pooling (permutation invariant)
        return self.head(global_feat)
```



In Class Activity: PointNet vs. PointTransformer

```
class PointTransformerLite(nn.Module):
    def __init__(self, num_classes=NUM_CLASSES, dim=8, k=12):
        super().__init__()
        self.k = k
        self.stem = nn.Sequential(
            nn.Linear(3, dim), nn.ReLU(),
            nn.Linear(dim, dim)
        )
        self.block = PointTransformerBlockLite(dim=dim, k=k)
        self.head = nn.Sequential(
            nn.Linear(2 * dim, 8), nn.ReLU(),
            nn.Dropout(0.2),
            nn.Linear(8, num_classes),
        )

    def forward(self, pos):
        idx = knn_indices(pos, self.k)  # Neighbor lookup
        x = self.stem(pos)  # Feature extraction
        x = self.block(x, pos, idx)  # Main attention block
        global_feat = torch.cat([x.mean(dim=1), x.max(dim=1).values], dim=-1)  # Global pooling
        return self.head(global_feat)
```



Global pooling

In Class Activity: PointNet vs. PointTransformer

```
class PointTransformerBlockLite(nn.Module):
```

```
    def __init__(self, dim, k=12):
```

```
        super().__init__()
```

```
        self.k = k
```

```
        self.to_q = nn.Linear(dim, dim, bias=False)
```

```
        self.to_k = nn.Linear(dim, dim, bias=False)
```

```
        self.to_v = nn.Linear(dim, dim, bias=False)
```

```
        self.pos_mlp = nn.Sequential(
            nn.Linear(3, dim), nn.ReLU(),
            nn.Linear(dim, dim)
        )
```

```
        self.attn_mlp = nn.Sequential(
            nn.Linear(dim, dim), nn.ReLU(),
            nn.Linear(dim, 1)
        )
```

```
        self.proj = nn.Linear(dim, dim)
```

```
        self.norm = nn.LayerNorm(dim)
```

```
    def forward(self, x, pos, idx):
```

```
        q = self.to_q(x) # [B, N, C]
```

```
        k = gather_neighbors(self.to_k(x), idx)
```

```
        v = gather_neighbors(self.to_v(x), idx)
```

```
        pos_n = gather_neighbors(pos, idx)
```

```
        rel = pos_n - pos.unsqueeze(2) # [B, N, K, 3]
```

```
        delta = self.pos_mlp(rel) # [B, N, K, C]
```

```
        attn_logits = self.attn_mlp(q.unsqueeze(2) - k + delta).squeeze(-1)
```

```
        attn = F.softmax(attn_logits / math.sqrt(x.shape[-1]), dim=-1)
```

```
        out = (attn.unsqueeze(-1) * (v + delta)).sum(dim=2)
```

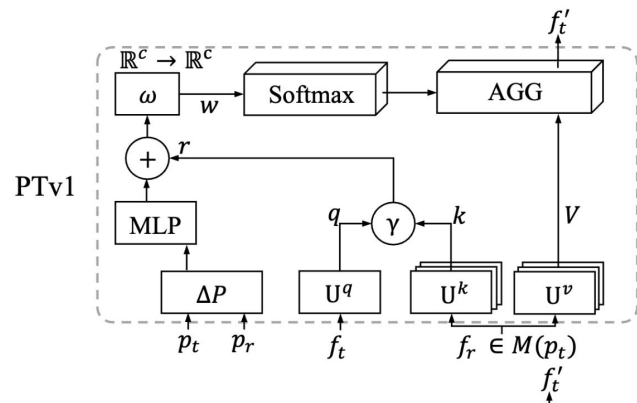
```
        out = self.proj(out)
```

```
        return self.norm(x + out)
```

Query
Key
Value

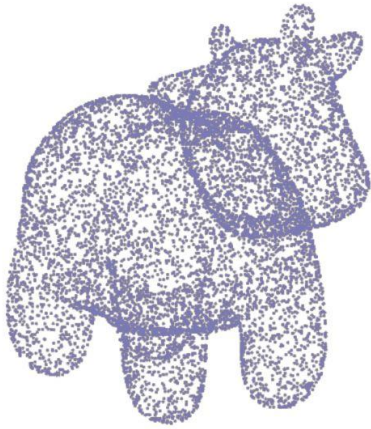
Position encoding

Attention



Demo: Google Colab

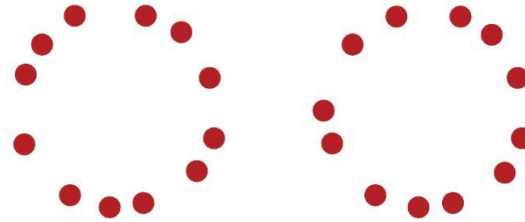
3D Point Cloud



$$\{\mathbf{p}_1, \mathbf{p}_2, \dots, \mathbf{p}_N\}$$

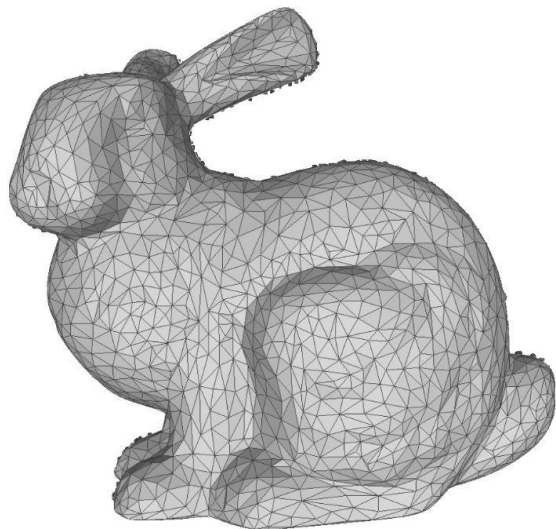
Unordered set of points

No explicit 'connectivity' information

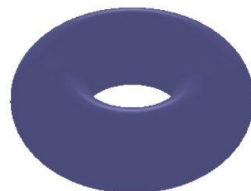
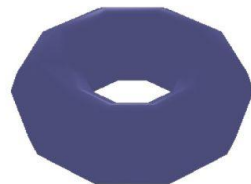


3D Mesh

Meshes



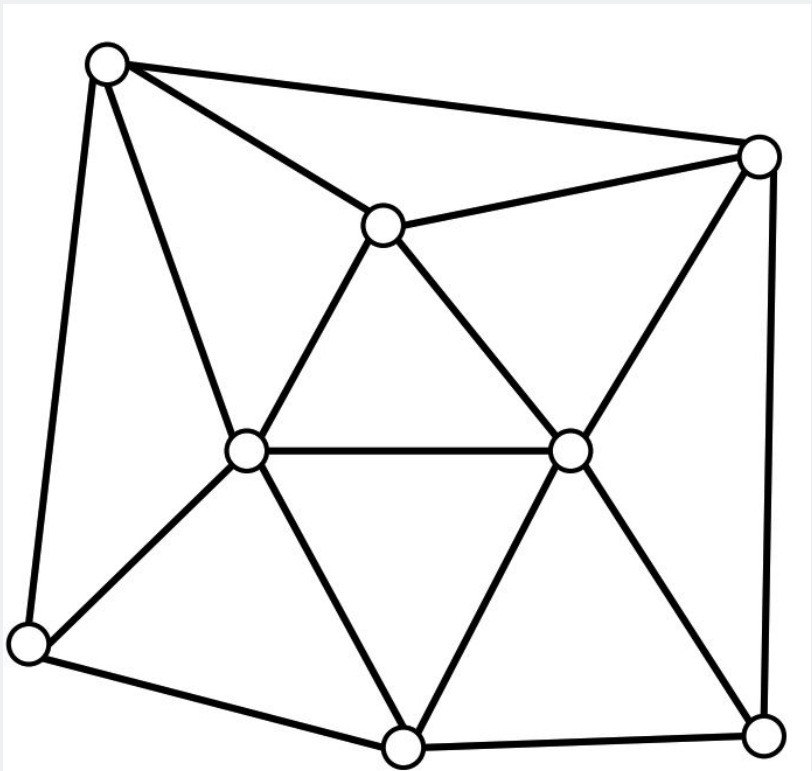
Vertices + Faces



Piecewise linear approximations of underlying surface

More elements allow better approximation

3D Shape Representations: Triangle Mesh



Represent a 3D shape as a set of triangles

Vertices: Set of V points in 3D space

Faces: Set of triangles over the vertices

(+) Standard representation for graphics

(+) Explicitly represents 3D shapes

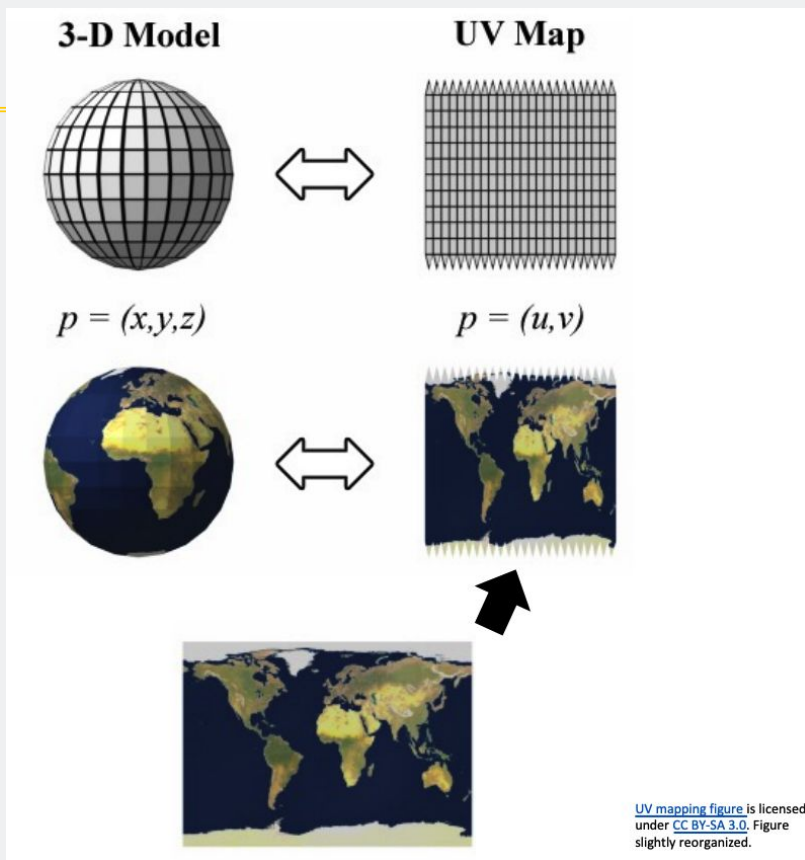
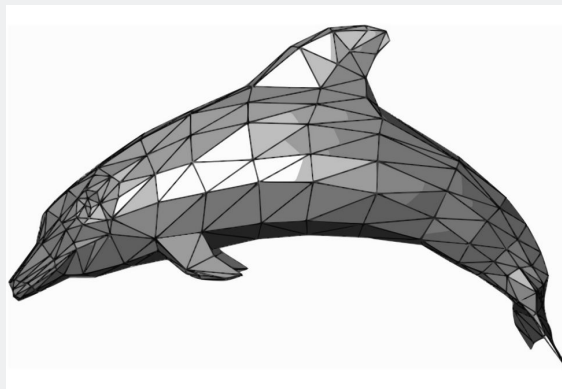
(+) Adaptive: Can represent flat surfaces very efficiently, can allocate more faces to areas with fine detail

(+) Can attach data on verts and interpolate over the whole surface: RGB colors, texture coordinates, normal vectors, etc.

Examples

<https://graphics.stanford.edu/data/3Dscanrep/>

“Stanford bunny”



SMPL

<https://smpl.is.tue.mpg.de/>

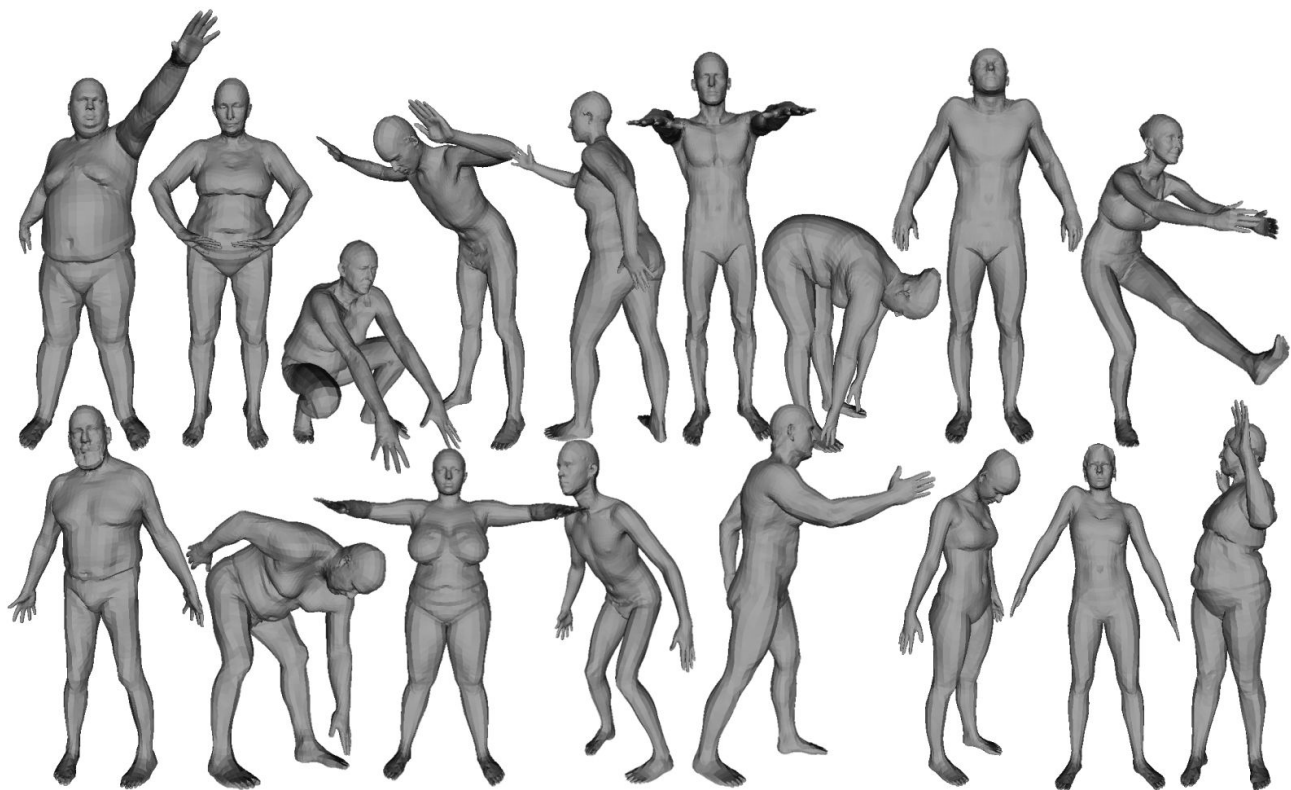
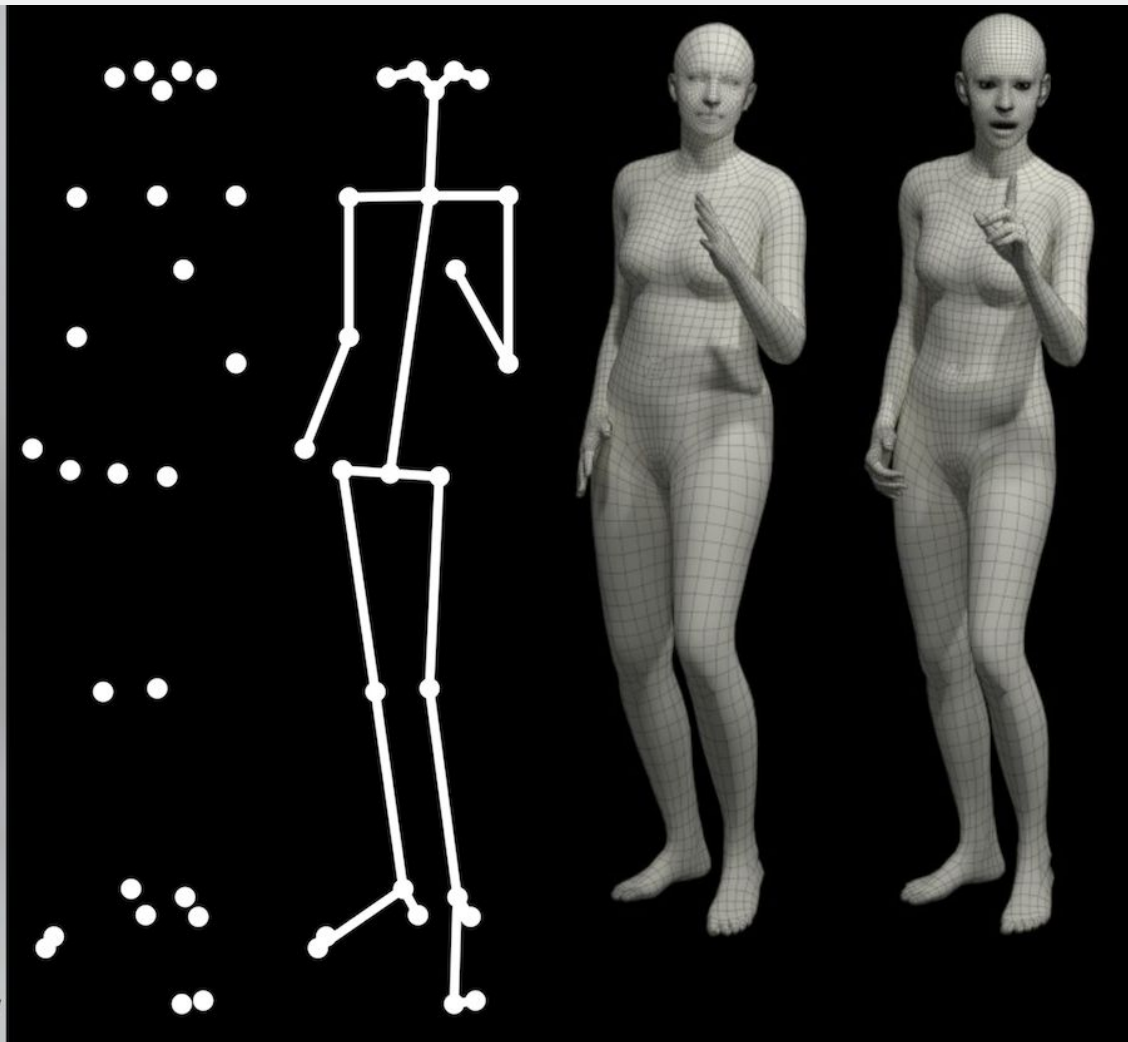


Figure 4: *Sample registrations from the multipose dataset.*

SMPL-X



Original image:
[https://www.gettyimages.com/
search/stack/546047069#](https://www.gettyimages.com/search/stack/546047069#)

<https://smpl-x.is.tue.mpg.de/>

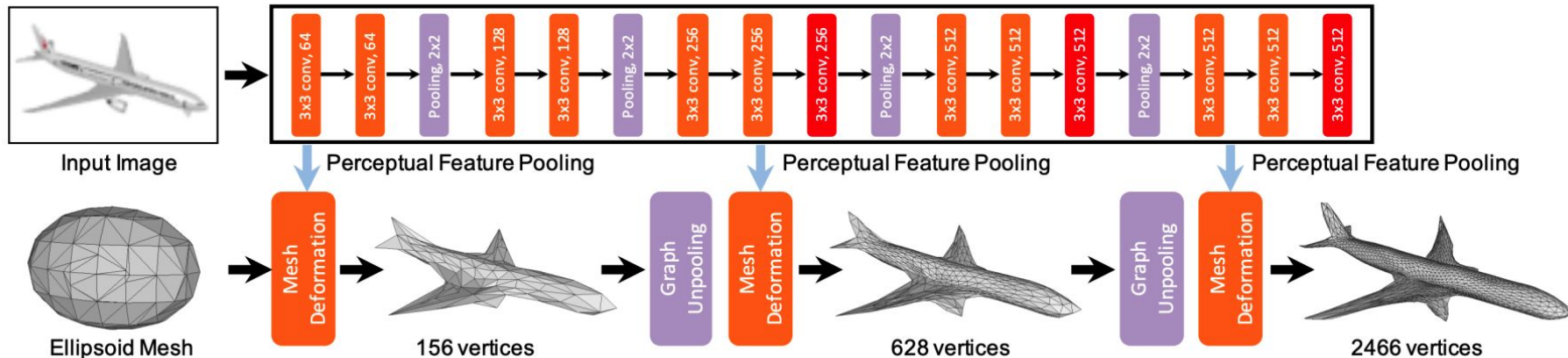
Predicting Meshes: Pixel2Mesh

Input: Single RGB
Image of an object

Key ideas:

- Iterative Refinement
- Graph Convolution
- Vertex Aligned-Features
- Chamfer Loss Function

Output: Triangle
mesh for the object



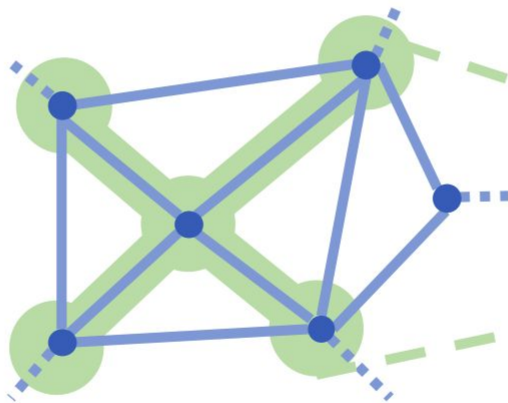
Graph Convolution

$$f'_i = W_0 f_i + \sum_{j \in N(i)} W_1 f_j$$

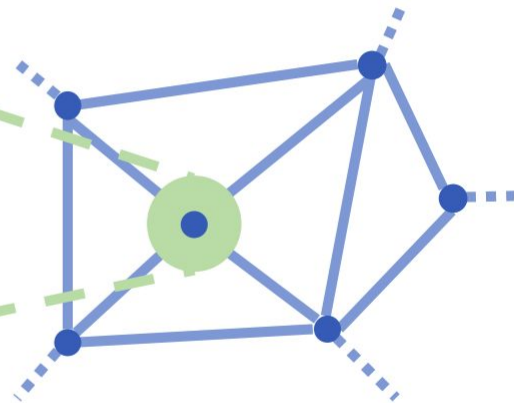
Vertex v_i has feature f_i

New feature f'_i for vertex v_i depends on feature of neighboring vertices $N(i)$

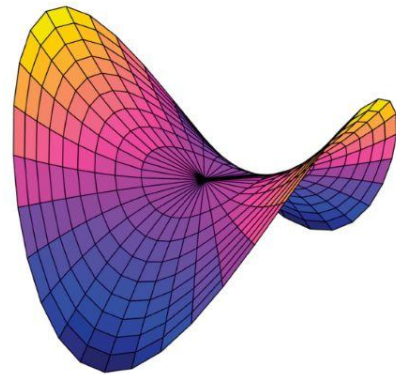
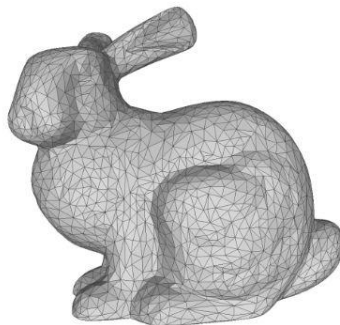
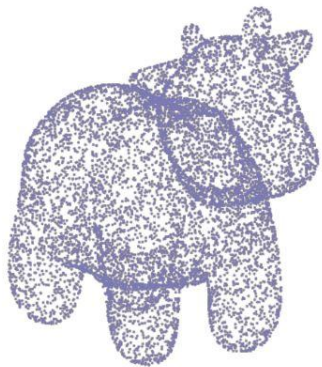
Use same weights W_0 and W_1 to compute all outputs



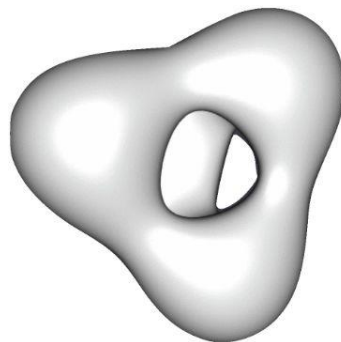
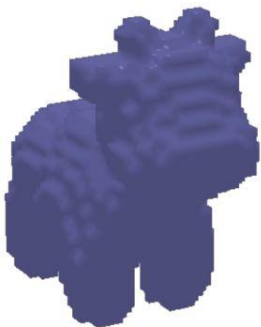
Input: Graph with a feature vector at each vertex



Output: New feature vector for each vertex

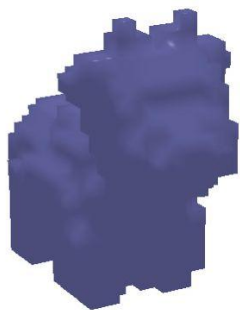


Boundary Representations for Shapes

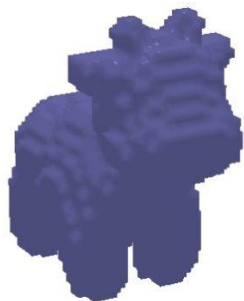


Volumetric Representations for Shapes

3D Voxels



32^3



64^3



128^3



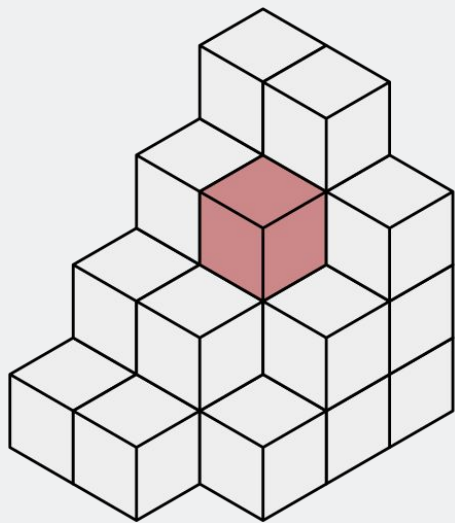
256^3



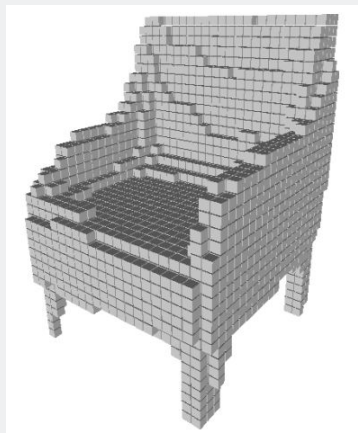
mesh

Computationally challenging to scale
resolution

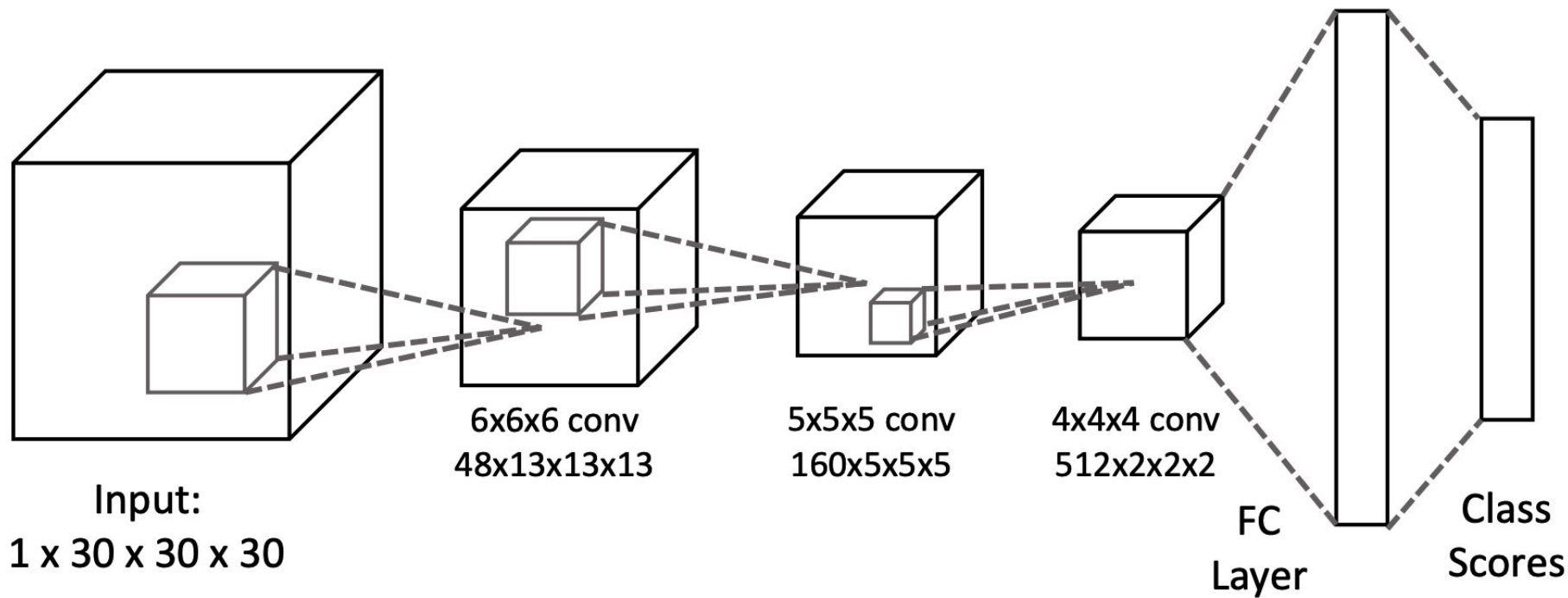
3D Shape Representations: Voxels



- Represent a shape with a $V \times V \times V$ grid of occupancies
- Just like segmentation masks in Mask R-CNN, but in 3D!
- (+) Conceptually simple: just a 3D grid!
- (-) Need high spatial resolution to capture fine structures
- (-) Scaling to high resolutions is nontrivial!



Learning on Voxel Inputs: 3D Convolution

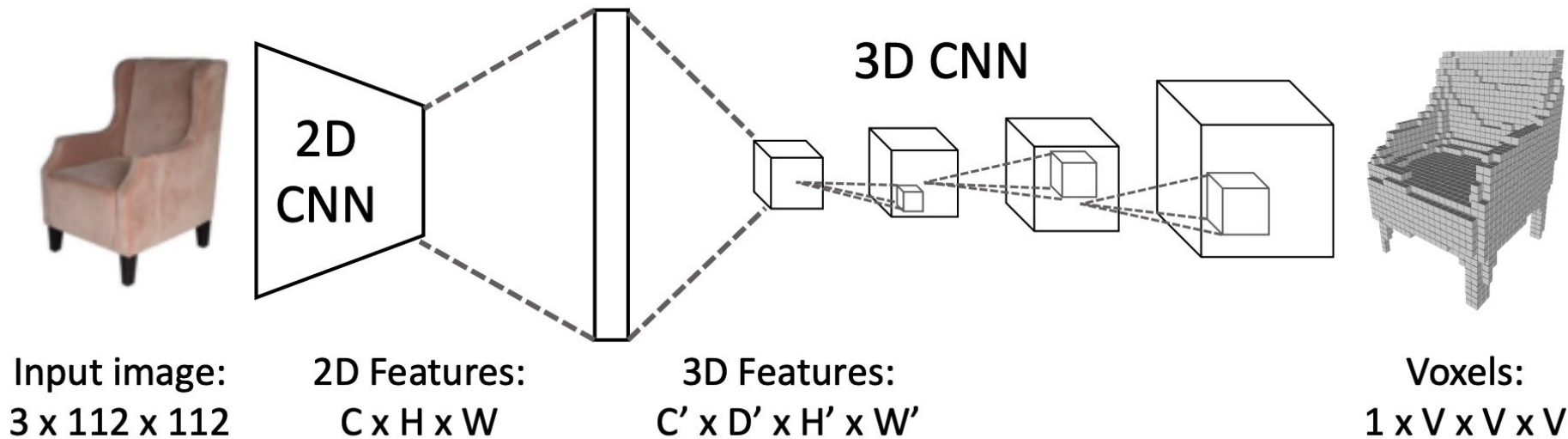


<https://arxiv.org/pdf/1406.5670>

<https://3dshapenets.cs.princeton.edu/>

Train with classification loss

Generating Voxel Shapes: 3D Convolution



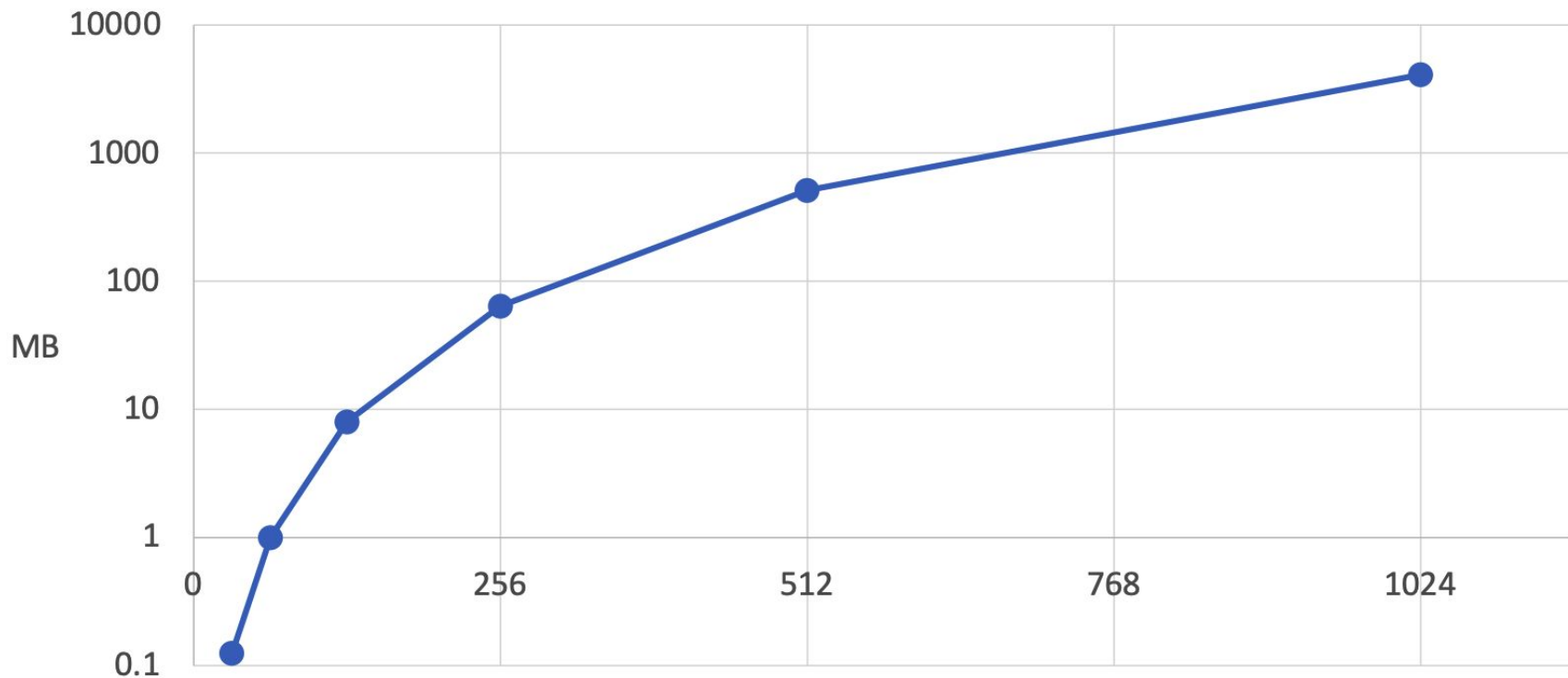
<https://arxiv.org/pdf/1604.00449>

Train with per-voxel cross-entropy loss

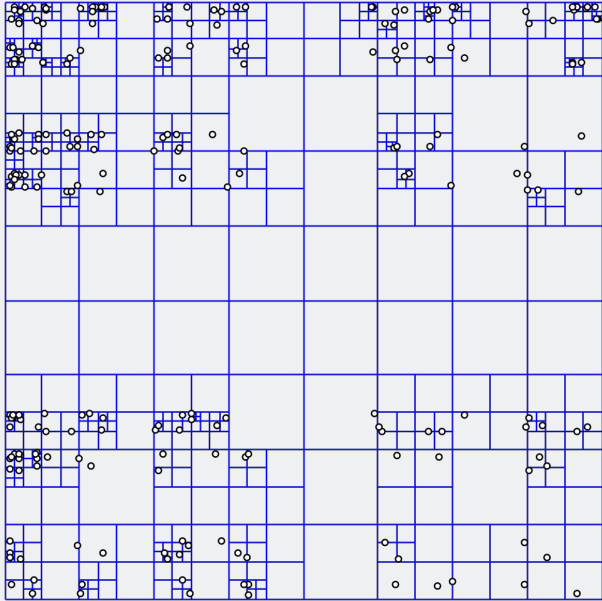
Voxel: Memory Usage

Storing 1024^3 voxel grid takes 4GB of memory!

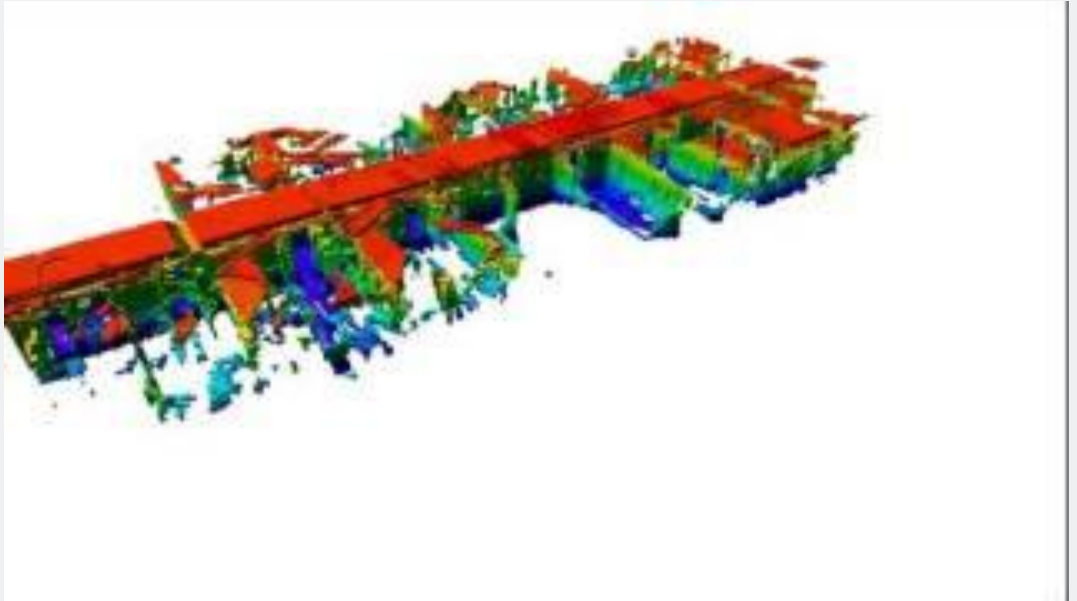
Voxel memory usage (V x V x V float32 numbers)



Scaling Voxels: Octrees



2D: quadtree



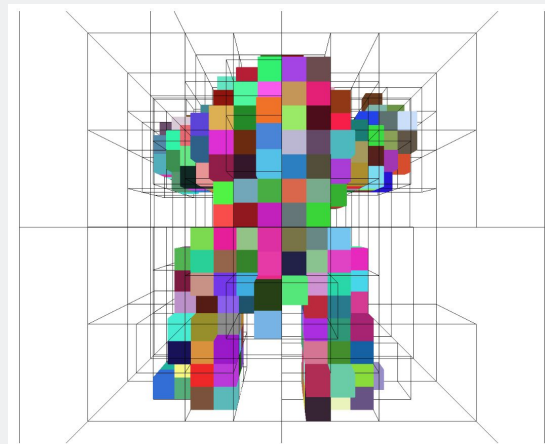
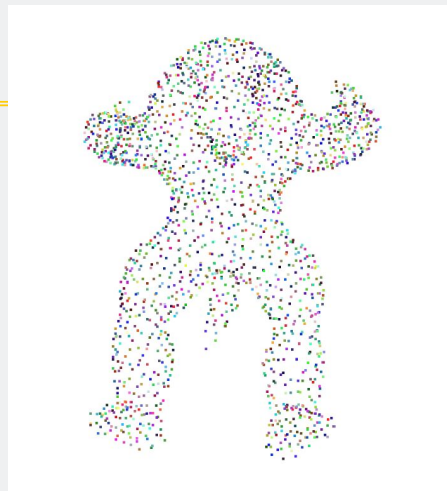
3D: octree

Demo: Open3D

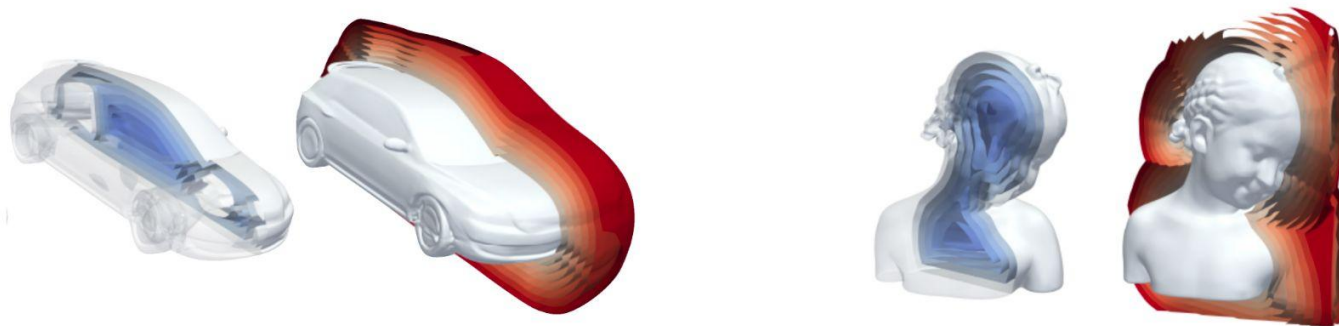
```
print('input')
N = 2000

armadillo = o3d.data.ArmadilloMesh()
mesh = o3d.io.read_triangle_mesh(armadillo.path)
pcd = mesh.sample_points_poisson_disk(N)
# fit to unit cube
pcd.scale(1 / np.max(pcd.get_max_bound() - pcd.get_min_bound()),
          center=pcd.get_center())
pcd.colors = o3d.utility.Vector3dVector(np.random.uniform(0, 1, size=(N, 3)))
o3d.visualization.draw_geometries([pcd])

print('octree division')
octree = o3d.geometry.Octree(max_depth=4)
octree.convert_from_point_cloud(pcd, size_expand=0.01)
o3d.visualization.draw_geometries([octree])
```



Implicit Surface



$$\{\mathbf{p} \mid f(\mathbf{p})\} = 0$$

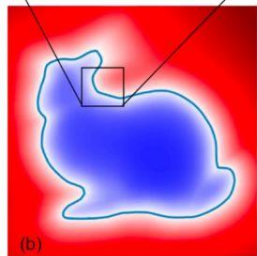
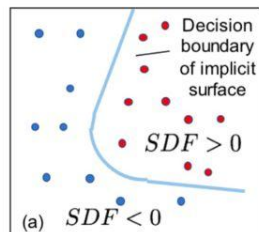
Additional condition that $\mathbf{f}$ represents
(signed) distance to surface from point $\mathbf{p}$

Image credits: IGR: Implicit Geometric Regularization for Learning Shapes. *Gropp et. al*

Signed Distance Function (SDF)

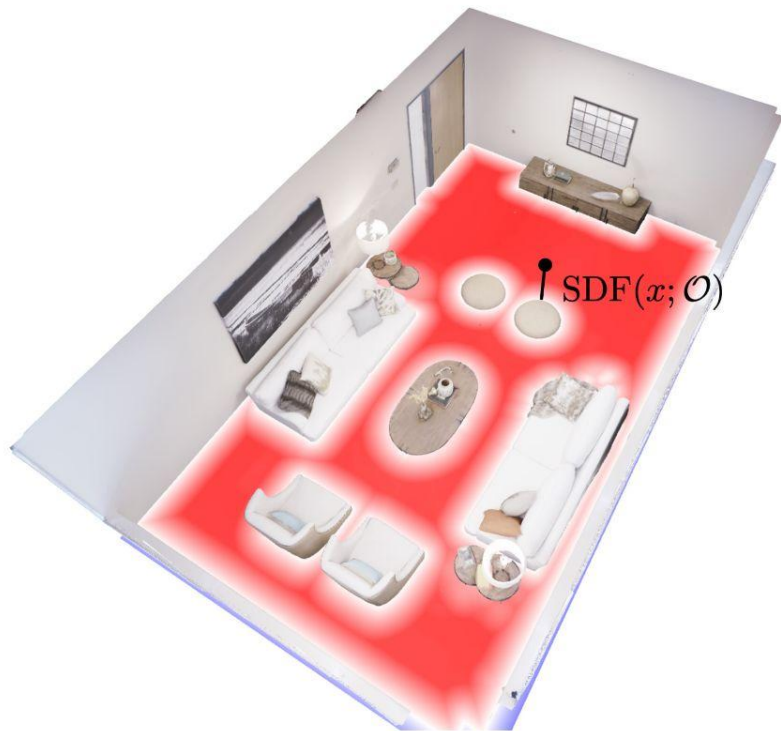
Definition 1. The *signed distance function* (SDF) of a set $\mathcal{O} \subset \mathbb{R}^n$ is a function $f : \mathbb{R}^n \rightarrow \mathbb{R}$ that measures the signed distance from a point $p \in \mathbb{R}^n$ to the set boundary $\partial\mathcal{O}$, defined as:

$$f_{\text{SDF}}(p; \mathcal{O}) \triangleq \begin{cases} \min_{y \in \partial\mathcal{O}} \|p - y\|_2, & p \notin \mathcal{O}, \\ -\min_{y \in \partial\mathcal{O}} \|p - y\|_2, & p \in \mathcal{O}. \end{cases} \quad (1)$$



3D SLAM with Signed Distance Function (SDF)

2

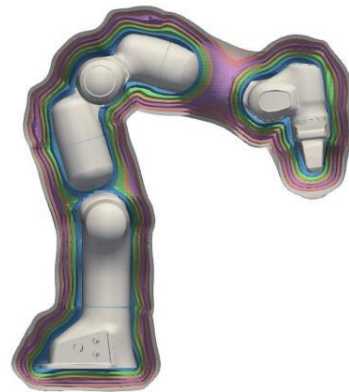


$$SDF(x; \mathcal{O}) = \begin{cases} \min_{y \in \partial \mathcal{O}} \|x - y\|_2, & x \notin \mathcal{O}, \\ -\min_{y \in \partial \mathcal{O}} \|x - y\|_2, & x \in \mathcal{O}. \end{cases}$$

Wide ranging applications in Robotics and 3D Vision



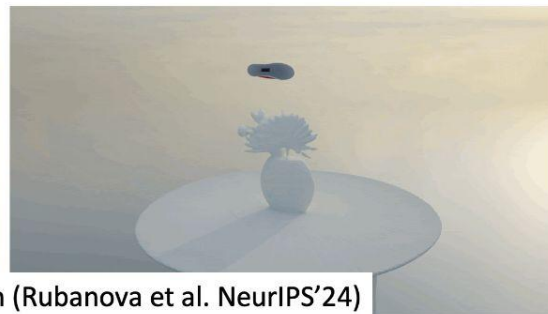
Navigation (Frey et al. RSS'23)



Manipulation (Li et al. ICRA'24)

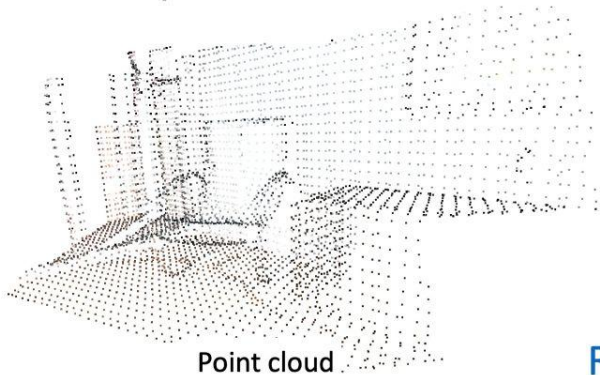


Rigid-Body Simulation (Rubanova et al. NeurIPS'24)

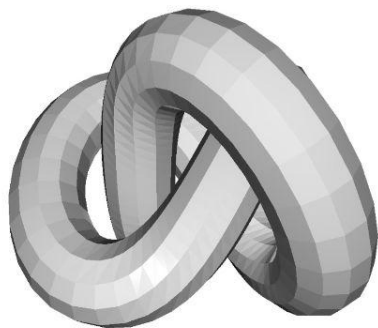


From SDF to Depth Images

- **Surface** representations



Rendering



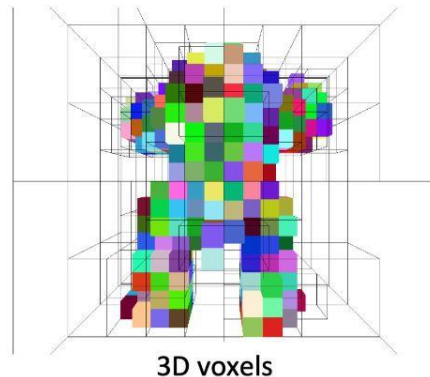
credits: Paul Bourke

An **image** that represents how far each pixel $\mathbf{p}$ is



Depth images

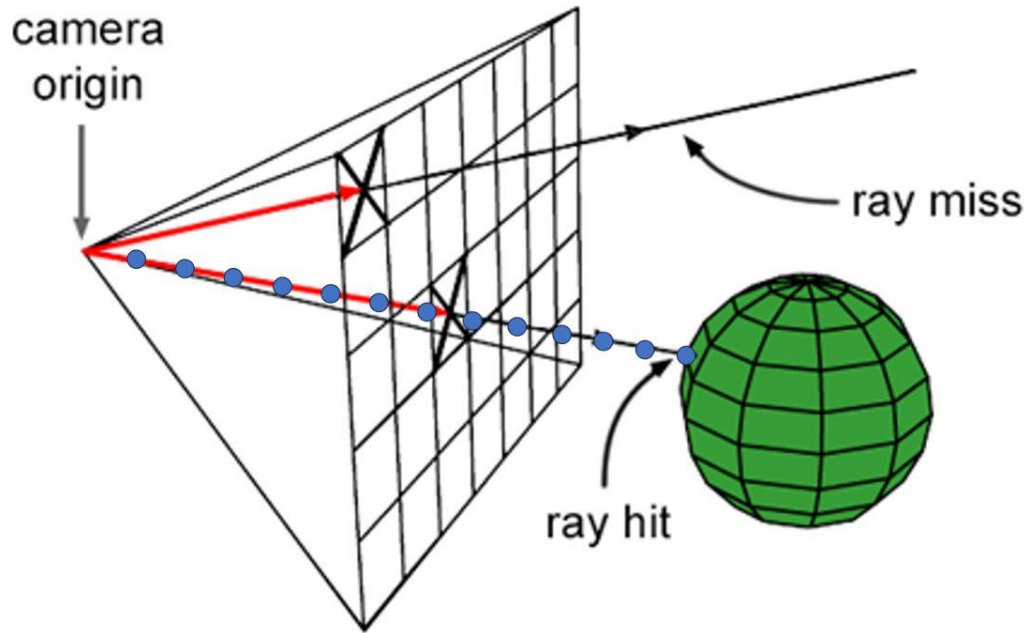
- **Volumetric** representations



Distance functions

Today's lecture

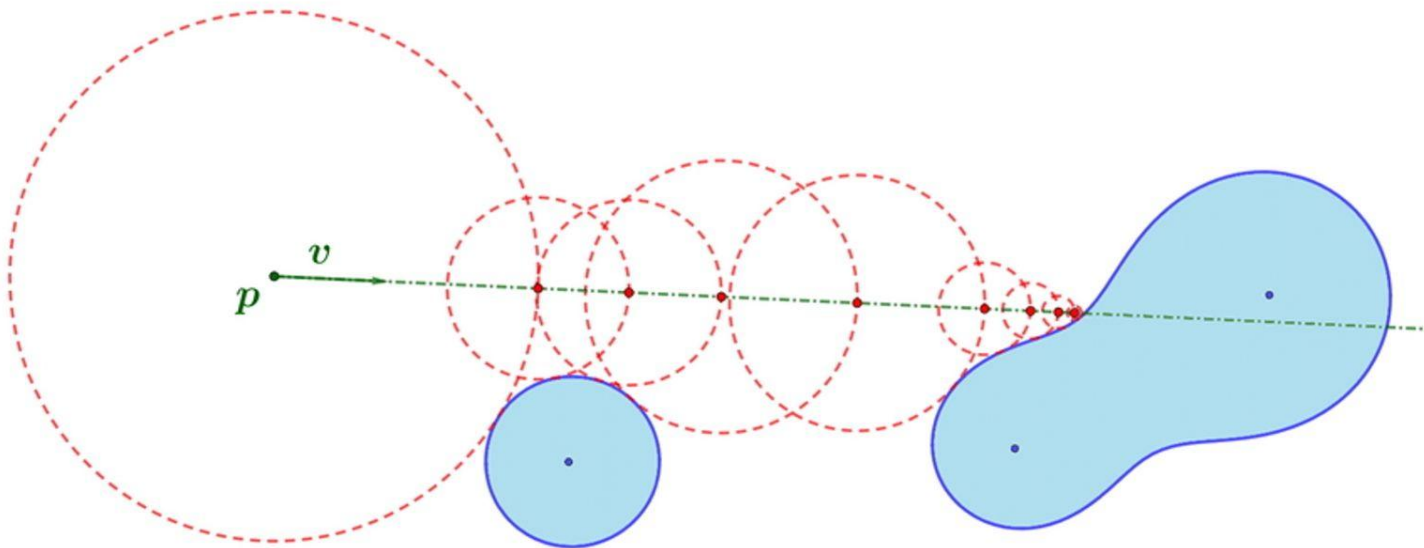
Rendering: Ray Marching



Can we use the property of SDF to speed up ray marching?

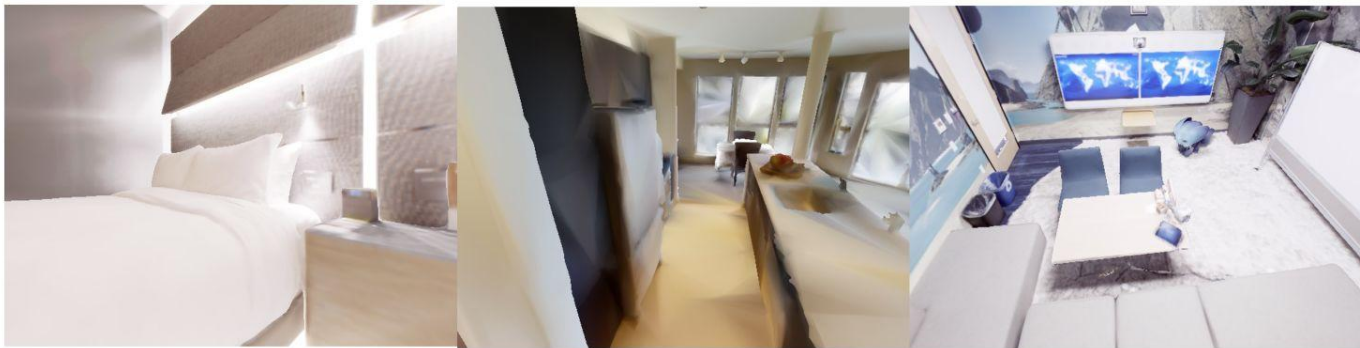
Rendering: Sphere Tracing

- Key idea: **SDF at any point gives minimum step size!**
- Further from surface $\rightarrow$ larger step size $\rightarrow$ faster rendering

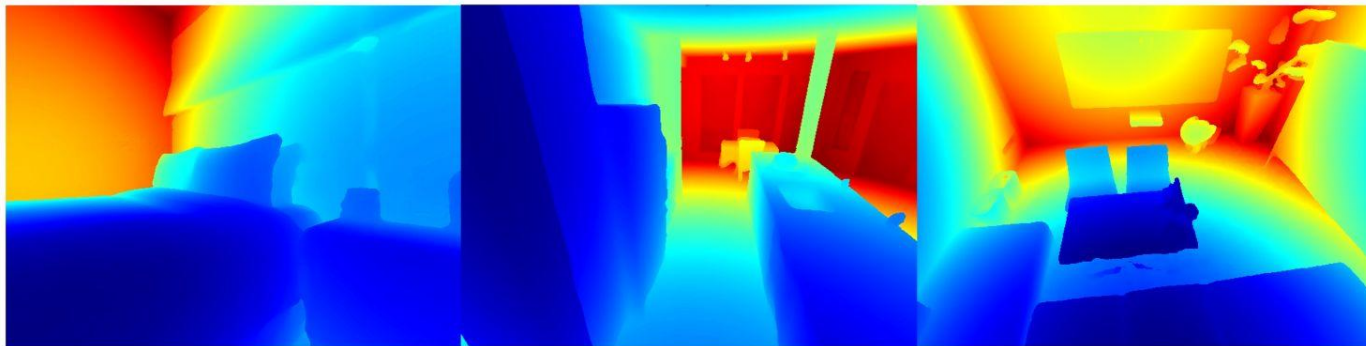


Rendering: Sphere Tracing

- Key idea: **SDF at any point gives minimum step size!**
- Further from surface $\rightarrow$ larger step size $\rightarrow$ faster rendering



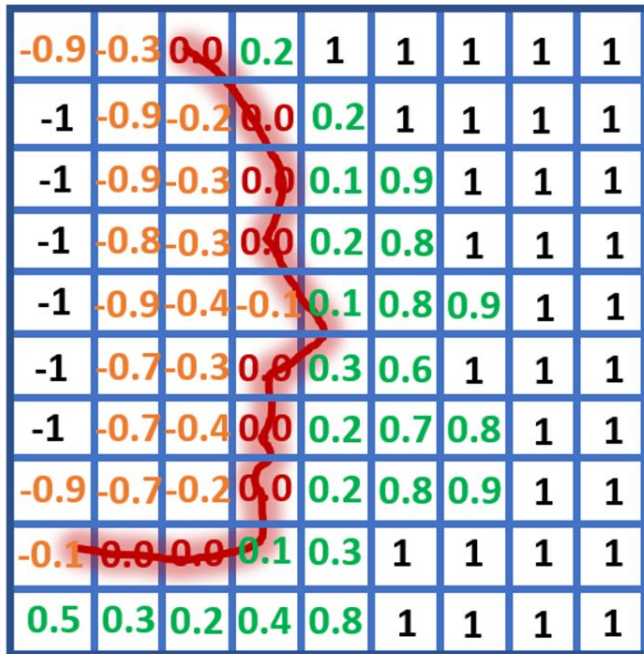
Input viewpoints (shown as RGB images)



Depth from sphere tracing

Storing SDF: Voxel Grid

But this is very expensive...



-0.9	-0.3	0.0	0.2	1	1	1	1	1	1
-1	-0.9	-0.2	0.0	0.2	1	1	1	1	1
-1	-0.9	-0.3	0.0	0.1	0.9	1	1	1	1
-1	-0.8	-0.3	0.0	0.2	0.8	1	1	1	1
-1	-0.9	-0.4	-0.1	0.1	0.8	0.9	1	1	1
-1	-0.7	-0.3	0.0	0.3	0.6	1	1	1	1
-1	-0.7	-0.4	0.0	0.2	0.7	0.8	1	1	1
-0.9	-0.7	-0.2	0.0	0.2	0.8	0.9	1	1	1
-0.1	-0.9	0.0	0.1	0.3	1	1	1	1	1
0.5	0.3	0.2	0.4	0.8	1	1	1	1	1

Image credit: Tim Cheng

From every raw depth image, we obtain a normalized and weighted TSDF:

$$F_{R_k}(p) = \begin{cases} \eta/tr, & \text{if } |\eta| \leq tr, \\ \text{sign}(\eta), & \text{otherwise.} \end{cases}$$

$$W_{R_k}(p) \propto \cos(\theta)/D(x)$$

Weight is higher if closer and viewed from a perpendicular viewpoint.

Next Time: Neural SDF (and More!)

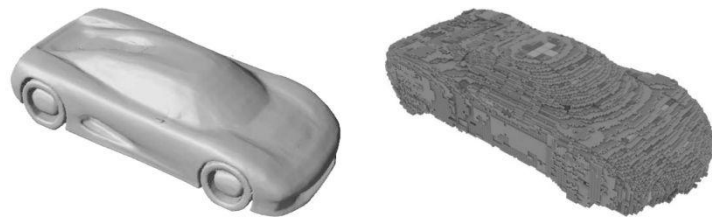
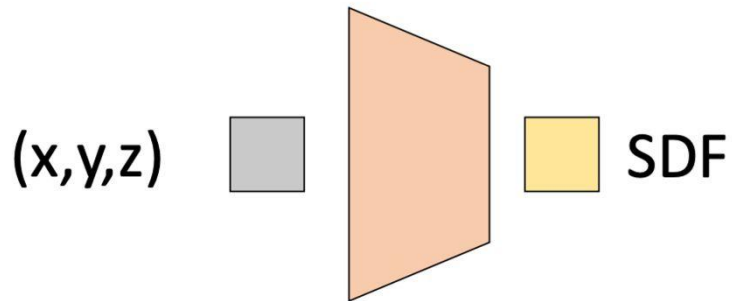


Figure 5: Compared to car shapes memorized using OGN [49] (right), our models (left) preserve details and render visually pleasing results as DeepSDF provides oriented surface normals.

Park et al. "DeepSDF: Learning continuous signed distance functions for shape representation." CVPR 2019.

A Primer on 3D Reconstruction

- **Input:** a set of overlapping images of the same scene from different viewpoints.
- **Goal:** estimate **camera parameters** and **3D scene structure** so the reconstructed scene explains the observed pixels.
- **Typical outputs:** camera poses, sparse 3D points, dense depth / point cloud / mesh.

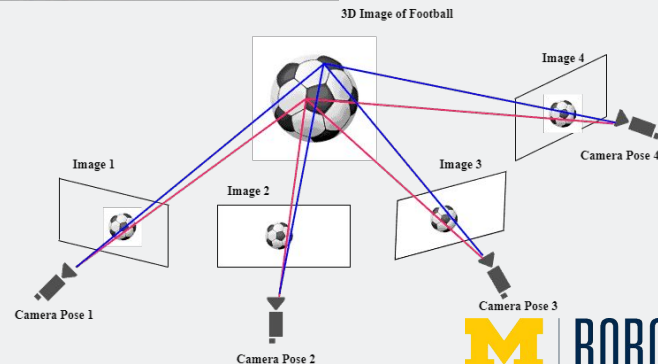
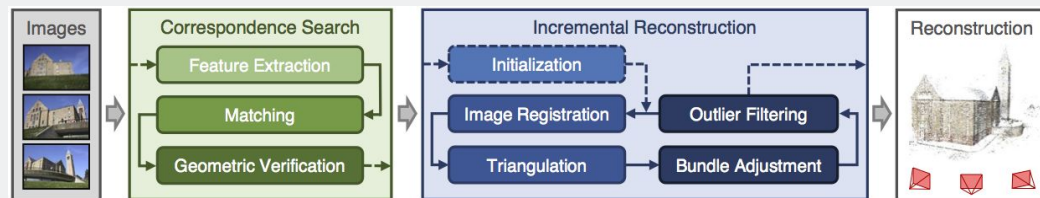


Figure 1. Result of Rome with 21K registered out of 75K images.

Structure from Motion (SfM)

Hand-Designed Modules

- **Feature extraction:** detect local features in each image.
- **Matching + geometric verification:** match features across images, then reject bad matches with epipolar geometry and RANSAC.
- **Incremental reconstruction:** initialize from a good image pair, register new cameras, triangulate points, repeat.



Modern Way: Feed-Forward Geometric Models



Figure 1. **VGGT** is a large feed-forward transformer with minimal 3D-inductive biases trained on a trove of 3D-annotated data. It accepts up to hundreds of images and predicts cameras, point maps, depth maps, and point tracks for all images at once in less than a second, which often outperforms optimization-based alternatives without further processing.

(Tentative) Apr 7 discussion: Try VGGT yourself!

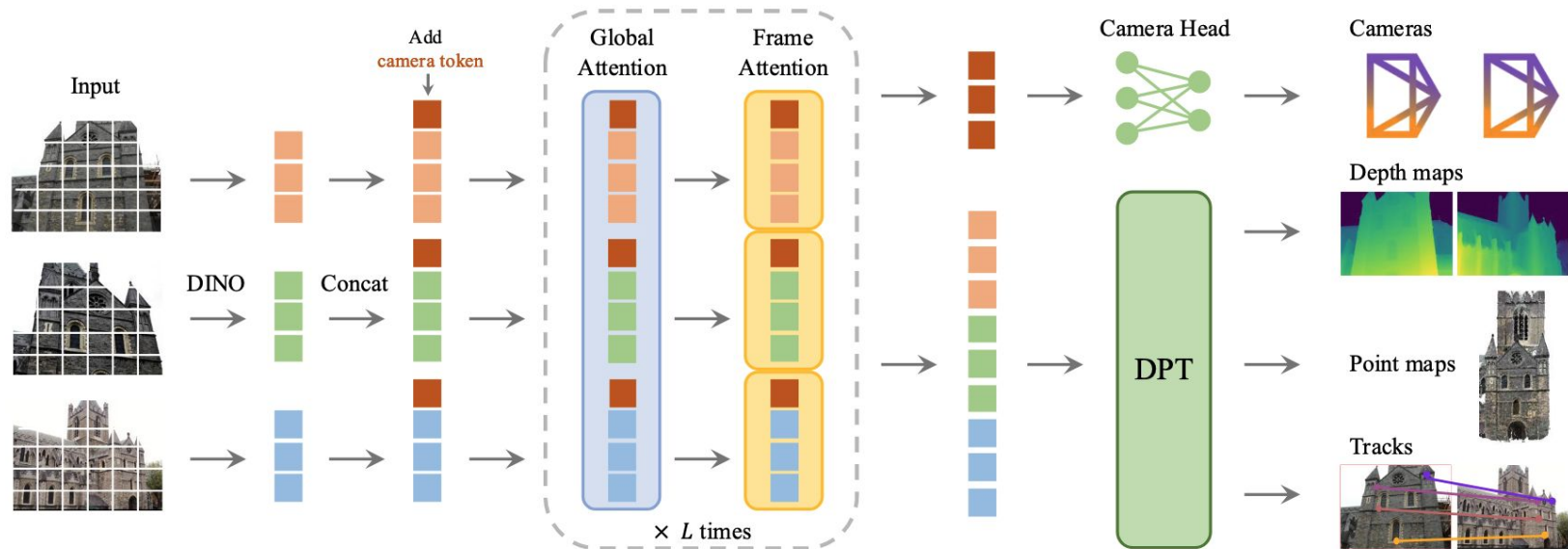
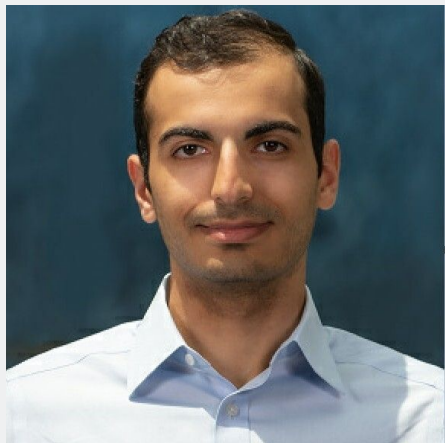


Figure 2. **Architecture Overview.** Our model first patchifies the input images into tokens by DINO, and appends camera tokens for camera prediction. It then alternates between frame-wise and global self attention layers. A camera head makes the final prediction for camera extrinsics and intrinsics, and a DPT [87] head for any dense output.

Guest Lecture (Apr 13)

Dominic Maggio (MIT) on VGGT for Robot Perception

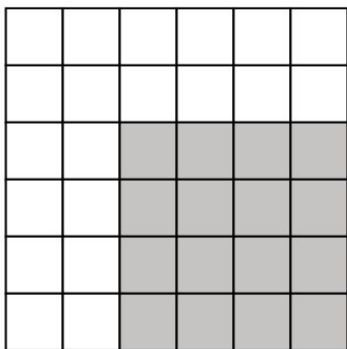


RECONSTRUCTION OF LARGE BARN

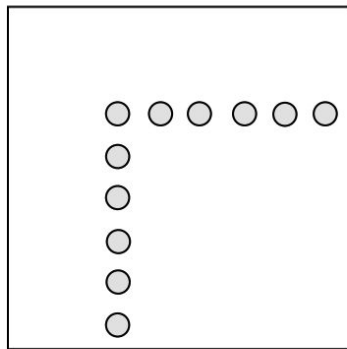
Summary



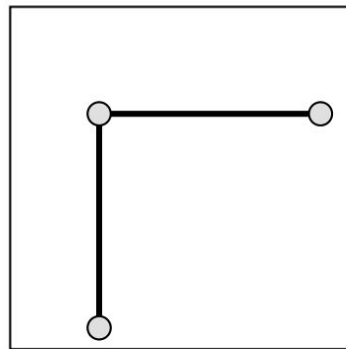
Depth
Map



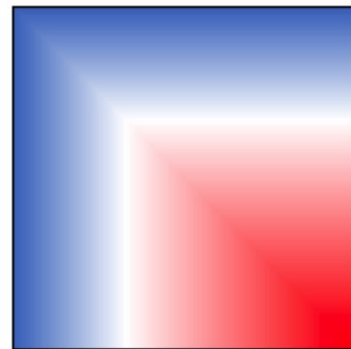
Voxel
Grid



Pointcloud



Mesh



Implicit
Surface

Reminder:

- **Final Project** “lightning talk” March 31, 2026 (tomorrow during discussion)